Assignment 2 (CO2) - Report

Operators, Decision-Making Statements and Looping Constructs

Questions 1 to 4: Precedence and Associativity, Short-Circuit Evaluation, Bitwise Operators, else-if Ladder vs switch

Student name	__Swamisetti Uma Sri_____
Register number	____1965260130_____
Course / Faculty	____C Programme-Surya Shan _____
GitHub repository	https://github.com/swamisettiumasri-debug/CSA0222_swamisetti-uma-sri-1965260130-/tree/main/ASSESSMENT%202
Compiler used for measurements	GCC 13.3.0 (Ubuntu 24.04). Compile command: `gcc -Wall -Wextra -o qN qN.c`
Screenshots	Student's own execution screenshots from an online C compiler (Figures 1 to 5)

Fill in the name, register number, faculty and the GitHub link before submission. The assignment states that a report without a working GitHub link is not evaluated.

Contents and marks

Section	Topic	Marks
Q1	Precedence, associativity and the discount bug (K4)	6
Q2	When does short-circuit evaluation matter? (K4)	4
Q3	Device-status register with bitwise operators, Complete the Code (K4)	6
Q4	else-if ladder or switch? Complete the Code (K4)	6
Appendix	Repository layout, build commands and reproducibility	-

Questions 5 to 7 are not part of this document.

Method used in every section

Each question follows the order required by the assignment: **Predict, Design, Implement, Measure, Analyse, Evaluate**. Predictions were worked out by hand from the C rules before any program was run. Raw counts (function calls, comparisons, loop iterations) are the primary evidence; the screenshots only confirm the final output. Counts are deterministic and can be reproduced exactly; the few timing figures depend on the machine and are used only as supporting observations.

Q1 - Precedence, Associativity and the Discount Bug

Jan Aushadhi Pharmacy billing. 6 marks, Bloom K4 (Analyse).

1.1 Problem analysis

Intended rule: the amount payable is price x quantity, **less 15% senior-citizen discount**, **plus 5% GST**, and then Rs 50 is deducted when the total is above Rs 1000. The assignment does not say which amount GST is charged on. I took the usual billing reading, **GST on the discounted amount**, and the corrected program (Figure 1) uses the same reading.

The developer's code:

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total;   /* Rs 50 off above Rs 1000 */ count
= count++ + 1;
```

Precedence and associativity rules that apply

Level (high to low)	Operators used here	Associativity
1 (highest)	( )	-
2	* / (same level)	left to right

3	+ - (same level)	left to right
4	>	left to right
5	?:	right to left
6 (lowest)	=	right to left

Precedence decides only *grouping*. It does not fix the order in which the three terms of the sum are evaluated; the compiler may evaluate them in any order, which does not matter here because the terms have no side effects.

1.2 Predictions before execution

Variables were assumed to be **int**, so every **/** truncates. Three bills were chosen: A (price 200, qty 6, senior), B (price 99, qty 3, senior) and C (price 50, qty 7, not senior).

Operator-by-operator evaluation of the original expression, Bill A (200, 6, senior = 1)

Step	Operation (grouping given by precedence and left-to-right associativity)	Result
1	price * qty (first * of term 1)	1200
2	(price * qty) * 15 (left-to-right, so multiplication before division)	18000
3	18000 / 100	180
4	180 * senior	180
5	(price * qty) * 5, then / 100 (term 3)	6000, then 60
6	1200 - 180 (left-to-right - and +)	1020
7	1020 + 60	1080
8	total = 1080 (assignment is last, lowest precedence)	1080
9	total > 1000, then ?: selects total - 50	1030

Associativity trap: because * and / are left-associative, `price*qty*15/100` multiplies by 15 *before* dividing by 100, which keeps the precision. Had the grouping been `price*qty*(15/100)`, then 15/100 = 0 in integer arithmetic and the discount would vanish (measured: 0).

Bill	Hand prediction, original expression (int)	Hand prediction, double variables	Intended (GST on discounted amount)
A (200, 6, 1)	total 1080, net 1030	1080.00 / 1030.00	gross 1200, discount 180, GST 51, total 1071, net 1021
B (99, 3, 1)	297; 4455/100 = 44; 1485/100 = 14; 297 - 44 + 14 = 267; net 267	297 - 44.55 + 14.85 = 267.30	discount 44.55, GST 12.6225, total 265.07
C (50, 7, 0)	350 - 0 + 1750/100 (= 17) = 367; net 367	350 + 17.5 = 367.50	GST 17.50, total 367.50

1.3 Implementation and observed results

Program `q1_compare.c` evaluates the original expression with int variables, with double variables, and the corrected calculation, for the three bills.

```
#include <stdio.h>
/* Q1: buggy single-expression billing vs corrected, three bills */
typedef struct { int price, qty, senior; } Bill;

int main(void)
{
    Bill b[3] = { {200, 6, 1}, {99, 3, 1}, {50, 7, 0} };
printf("%-12s %-10s %-10s %-10s %-10s %-10s\n",
          "Bill(p,q,s)", "int total", "int net", "dbl total", "dbl net", "right tot", "right net");
for (int i = 0; i < 3; i++) {
        int price = b[i].price, qty = b[i].qty, senior = b[i].senior;

        /* original expression, all-int variables (integer division truncates) */
int itotal = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
int inet   = itotal > 1000 ? itotal - 50 : itotal;

        /* original expression, double variables (no truncation, precedence bug stays) */
double dp = price, dq = qty, ds = senior;
```

```
        double dtotal = dp * dq - dp * dq * 15 / 100 * ds + dp * dq * 5 / 100;
double dnet   = dtotal > 1000 ? dtotal - 50 : dtotal;

        /* corrected: GST on the discounted amount */
float gross = (float)price * qty;
        float discount = senior ? gross * 15 / 100 : 0;
float tax = (gross - discount) * 5 / 100;          float
total = gross - discount + tax;          float net =
(total > 1000) ? total - 50 : total;

        printf("(%d,%d,%d)%*s %-10d %-10d %-10.2f %-10.2f %-10.2f %-10.2f\n",
price, qty, senior, 4, "", itotal, inet, dtotal, dnet, total, net);
    }
return 0; }
```

```
Bill(p,q,s)  int total  int net    dbl total  dbl net    right tot  right net
(200,6,1)     1080       1030       1080.00    1030.00    1071.00    1021.00
(99,3,1)     267        267        267.30     267.30     265.07     265.07
(50,7,0)     367        367        367.50     367.50     367.50     367.50
```

Bill	Predicted total/net (int)	Observed int total/net	Observed double total	Intended total/net	Error of original	Verdict
A	1080 / 1030	1080 / 1030	1080.00	1071.00 / 1021.00	+9.00 (Rs 9 overcharged)	WRONG
B	267 / 267	267 / 267	267.30	265.07 / 265.07	+1.93 (int)	WRONG
C	367 / 367	367 / 367	367.50	367.50 / 367.50	-0.50 (int) / 0.00 (double)	wrong in int only

All predictions matched the observed output exactly. The step-by-step trace program `q1_steps.c` reproduced the Bill A table above:

```
T1=price*qty=1200
T2: price*qty=1200, *15=18000, /100=180, *senior=180
T3: price*qty=1200, *5=6000, /100=60
T1 - T2 = 1020, + T3 = 1080
price*qty*(15/100)*senior = 0   (15/100 is 0 in integer arithmetic)
```

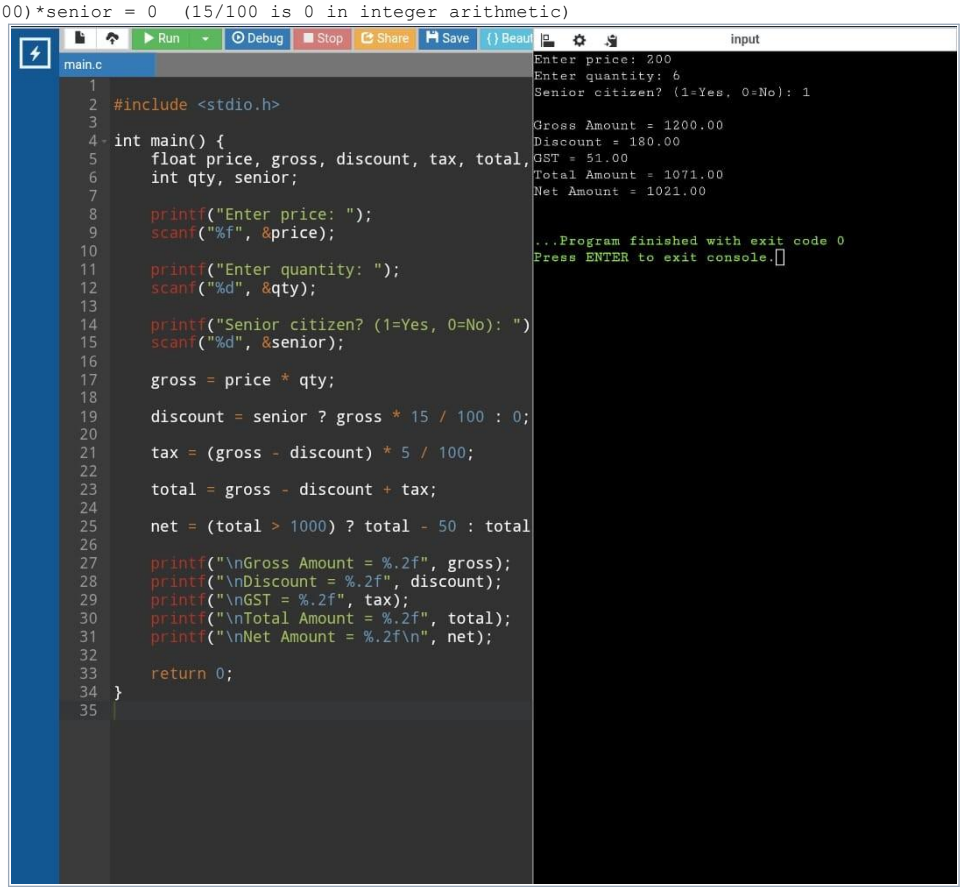


Figure 1 - Student's execution of the corrected program (price 200, qty 6, senior = 1): gross 1200, discount 180, GST 51, total 1071, net 1021. This matches the hand-calculated intended values.

Expressions that differ from what the pharmacist intended

- **GST base (logic error).** The original charges 5% on `price*qty` (the gross amount) instead of the discounted amount. For a senior citizen this overcharges by 5% of the discount: Rs 9 on Bill A (5% of 180).

- **Integer division.** Each `/ 100` truncates before the terms are combined: 4455/100 gives 44 (true 44.55), 1485/100 gives 14 (true 14.85), 1750/100 gives 17 (true 17.50). Money amounts are lost in paise, up to Rs 1 per term.
- **Net amount.** The Rs 50 rule is applied to the wrong total, so the error is carried into `net` (Bill A: 1030 instead of 1021). A bill near Rs 1000 could even be put on the wrong side of the threshold.
- **Third statement.** `count = count++ + 1;` has undefined behaviour (next section).

1.4 Inquiry: why is count = count++ + 1 undefined behaviour?

In this statement the object *count* is modified twice, once by the post-increment and once by the assignment, and its value is also read for the addition. C11 (6.5p2) says that if a side effect on a scalar object is **unsequenced** relative to another side effect on the same object, or to a value computation using that object, the behaviour is undefined. The assignment is sequenced after the value computations of its operands, but nothing orders the store done by `++` against the store done by `=`. A compiler may therefore produce count+1, produce the old value, or anything else.

What GCC does with -Wall. It compiles the file but prints a warning; it does not stop:

```
q1_count.c: In function 'main': q1_count.c:6:11: warning: operation on 'count' may
be undefined [-Wsequence-point]
    6 |     count = count++ + 1;
      |     ~~~~~~^~~~~~~~~~~~
```

With count = 5, the program printed **count = 6** at -O0, -O2 and -O3 on GCC 13.3. That agreement is only luck of this compiler version; the standard does not guarantee it. The intent is simply to add 1, so the correct statements are `count++;` or `count = count + 1;`.

1.5 Design decision: the corrected calculation

```
gross    = price * qty;
discount = senior ? gross * 15 / 100 : 0;      /* multiply first, divide last */ tax
= (gross - discount) * 5 / 100;       /* the one place that needs brackets */ total
= gross - discount + tax; net      = (total > 1000) ? total - 50 : total;
```

Decision: use both intermediate variables and brackets.

- Intermediate variables (gross, discount, tax) give each business rule its own line. Each value can be printed (as Figure 1 does) and checked against the hand calculation, so the GST-base bug becomes visible.
- Brackets are used in exactly one place, `(gross - discount)`, where the intended grouping differs from what precedence would give. Brackets everywhere would clutter the code; none at all caused the bug.
- The order `gross * 15 / 100` is kept: multiplying before dividing preserves precision.
- Money should not use int division. The submitted program uses float; double (or integer paise) is safer for real billing because float keeps only about 7 significant digits.

1.6 Alternative concept and complexity analysis

Alternative: replace the single long expression by several statements with intermediate variables, which is exactly the corrected program.

Aspect	Single expression (original)	Intermediate variables (corrected)
Arithmetic operations	10 (price*qty is computed three times; the discount term has 4 operators, the GST term 3, the combination 2)	8 arithmetic operations plus one conditional (gross: 1, discount: 2, tax: 3, total: 2)
Time	O(1), 10 operations	O(1), 8 operations. Saves repeated price*qty. With -O2 the compiler usually removes the repetition anyway, so the real saving is small.
Extra space	none declared	3 temporaries (gross, discount, tax) = 12 bytes as float, O(1); normally kept in registers
Correctness	GST base wrong; hard to see	Each step can be tested separately; bug removed
Readability	Needs a precedence table to read	One rule per line, self-documenting

Conclusion: time is O(1) in both versions and the extra space is negligible. Intermediate variables win on correctness and readability, and slightly on operation count.

Pseudocode (corrected)

```
READ price, qty, senior
gross    <- price * qty
```

```
IF senior THEN discount <- gross * 15 / 100 ELSE discount <- 0
tax      <- (gross - discount) * 5 / 100 total    <- gross -
discount + tax
IF total > 1000 THEN net <- total - 50 ELSE net <- total
PRINT gross, discount, tax, total, net
```

Q2 - When Does Short-Circuit Evaluation Matter?

Railway senior-citizen concession. 4 marks, Bloom K4 (Analyse).

2.1 Problem analysis and experiment design

Rule used in the experiments: **concession = (senior AND eligible class) OR (disability-card holder AND card verifies)**. The function `verify_card()` stands for the slow server call; every call increments a counter. Passenger data are generated by a fixed pseudo-random generator, so every version sees exactly the same 1000 passengers.

Generated data (target 25% senior, 40% eligible class, 8% card holders). Actual counts: senior = 248, eligible = 405, card holders = 89, senior AND eligible = 99.

Figure 2 is the student's interactive single-passenger program using the rule `(senior && eligible) || disability || verify_card()`. With inputs 1, 1, 0 it printed "Concession Granted" and **Verification calls = 0**: the left side of the `||` was already true, so the slow function was never called.

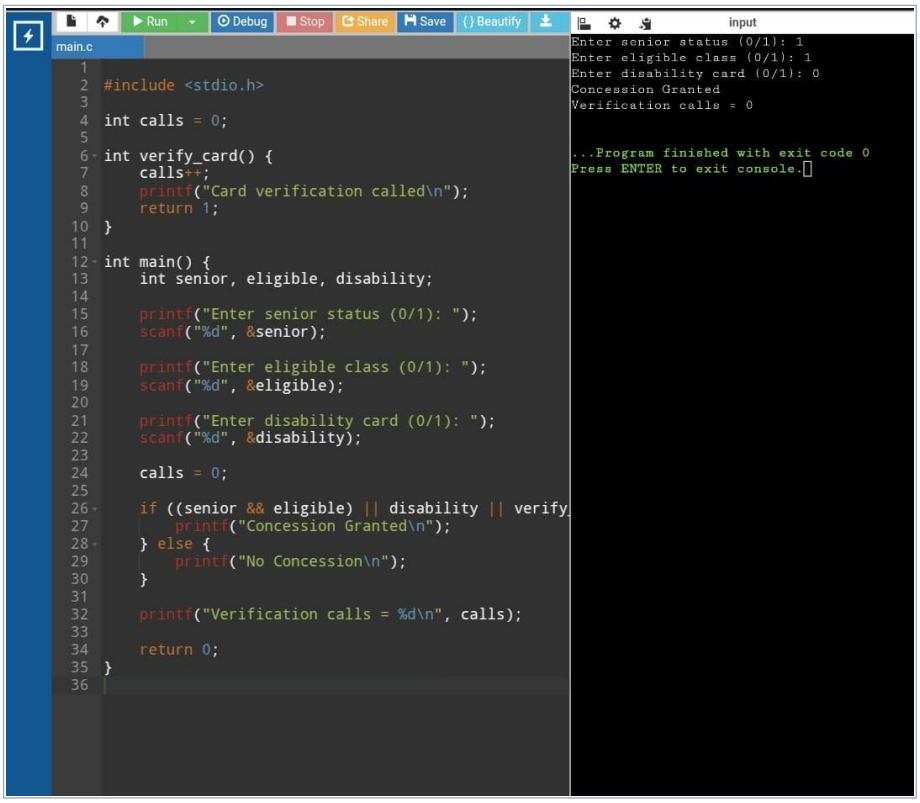


Figure 2 - Student's execution: the slow function is skipped (calls = 0) because (senior && eligible) is already true.

2.2 Experiment 1: when is the second operand not evaluated?

Each side-effect assignment below shows whether the right operand ran (a flag changes to 1 only if it was evaluated).

```
0 && (a=1): result=0 a=0
1 && (b=1): result=1 b=1
1 || (c=1): result=1 c=0
0 || (d=1): result=1 d=1
0 &  (e=1): result=0 e=1
1 |  (f=1): result=1 f=1
2 & 1 = 0   but   2 && 1 = 1
```

Expression	Right operand evaluated?	Reason
0 && (a = 1)	No (a stays 0)	Left is false, so the && result is already 0
1 && (b = 1)	Yes	Left is true, the right operand decides the result
1 || (c = 1)	No (c stays 0)	Left is true, so the || result is already 1

0 || (d = 1)	Yes	Left is false, the right operand decides
0 & (e = 1) and 1 | (f = 1)	Always (e = f = 1)	& and | are bitwise operators: no short-circuit
2 & 1 versus 2 && 1	-	2 & 1 = 0 (bits 10 and 01 share nothing) but 2 && 1 = 1. On conditions that are not exactly 0 or 1, & gives wrong answers.

Difference between & and &&: && is a logical operator that guarantees left-to-right order, stops as soon as the result is known, and treats any non-zero value as true. & is a bitwise operator on the bits of both operands; it always evaluates both and gives no ordering guarantee.

2.3 Experiment 2: order of conditions and the number of slow calls (1000 passengers)

```
V1   (S && E) || (C && verify())      cheap check first, card guard before the slow call
V2   (C && verify()) || (S && E)      card guard first
V3   (verify() && C) || (S && E)      slow call written first
V4   (S & E) | (C & verify())         non-short-circuit operators
V5  nested if (same logic as V1)
V6   (E && S) || (C && verify())      cheap conditions swapped
```

```
V1 (S&&E)||(C&&verify)            slow calls=  78  granted=177  audit entries=  78  senior/elig evals=1000/248 (sum 1248)
V2 (C&&verify)||(S&&E)            slow calls=  89  granted=177  audit entries=  89  senior/elig evals=911/227 (sum 1138)
V3 (verify&&C)||(S&&E)            slow calls=1000  granted=177  audit entries=1000  senior/elig evals=911/227 (sum 1138)
V4 (S&E)|(C&verify) bitwise       slow calls=1000  granted=177  audit entries=1000  senior/elig evals=1000/1000 (sum 2000)
V5 nested if (same as V1)         slow calls=  78  granted=177  audit entries=  78  senior/elig evals=1000/248 (sum 1248)
V6 (E&&S)||(C&&verify)            slow calls=  78  granted=177  audit entries=  78  senior/elig evals=405/1000 (sum 1405)
```

Version	Slow calls	Granted	Cheap evaluations (senior + eligible)	Weighted cost (slow call = 1000 cheap)	Relative to V1
V1	78	177	1248	79,248	1.00
V2	89	177	1138	90,138	1.14
V3	1000	177	1138	1,001,138	12.63
V4	1000	177	2000	1,002,000	12.64
V5	78	177	1248	79,248	1.00
V6	78	177	1405	79,405	1.00

The weight of 1000 cheap evaluations per slow call is an assumption made for illustration; the slow-call counts themselves are measured.

Observations

- All six versions grant the same 177 concessions, so they are logically equivalent. Only the cost differs.
- **V3 and V4 call the slow function 1000 times** (once per passenger): V3 because verify() is written first, V4 because & and | evaluate both sides.
- **V2 calls it 89 times**, exactly the number of card holders, because C guards the slow call.
- **V1 calls it 78 times**: the 89 card holders minus the 11 who already qualified through (senior AND eligible), which is tested first and skips the right side of the ||.
- **V5 (nested if) equals V1 (78 = 78)**: short-circuit operators are just a compact form of the same branching.
- **V1 versus V6:** swapping the cheap conditions does not change the slow calls (78), but cheap evaluations rise from 1248 to 1405. Senior is true for 24.8% of passengers and eligible for 40.5%, so testing senior first stops earlier more often.

2.4 Experiment 3: effect of how many passengers hold a card

```
card%     V1     V2     V3     V4     1
8       8      1000   1000
8         78     89     1000   1000
25        229    251    1000   1000
50        474    517    1000   1000
90        814    893    1000   1000
```

V3 and V4 stay at 1000 whatever the data. V1 and V2 grow with the proportion of card holders and never exceed it. Approximate model: V1 makes n x p x (1 - p) calls, V2 makes n x p , V3 and V4 make

card S and E card

n. For 8% card holders: 1000 x 0.089 x 0.901 = 80 (measured 78) and 89 (measured 89).

2.5 When should the cheapest or most-likely-false condition come first?

- **For A && B:** put first the condition that is cheap and likely to be false, because then B is skipped most often. Expected cost = cost(A) + P(A true) x cost(B). Here the card check (false for 91% of passengers, and very cheap) must guard the slow call.
- **For A || B:** put first the condition that is cheap and likely to be *true*, because then B is skipped. This is why (senior && eligible) before the card branch saves 11 slow calls.
- **Order is irrelevant** when both operands are side-effect-free, cost the same and have the same probability, or when both must be evaluated anyway (& and |). It is **never** irrelevant when the first operand guards the second, as in `p != NULL && p->x`.

2.6 Changed condition: the verification function now writes an audit log

In the experiment every call to verify_card() also appends the passenger to an audit log. The audit entries equal the slow calls: V1 = 78, V2 = 89, V3 and V4 = 1000. Re-evaluation:

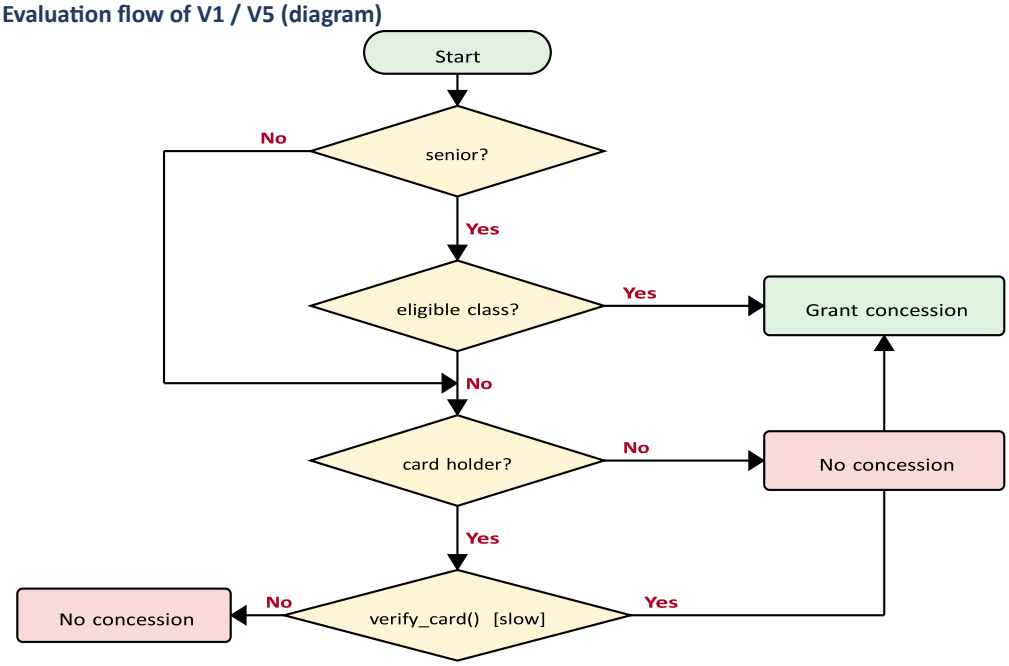
- **V3 and V4 become clearly unacceptable:** they log 911 passengers who hold no card. That wastes storage and one log write per call, and records people who were never verified.
- **V1 versus V2 now depends on the audit rule.** If the log must contain every card holder who presented a card, V1 is wrong because it silently skips the 11 card holders already granted by (senior AND eligible); the log would hold 78 instead of 89 entries. **Recommended order for that policy: V2 (card guard first)**, at the cost of 11 more slow calls (1.1% of passengers).
- If the log only needs the verifications that were actually required for a decision, V1 remains best (fewest calls).
- General rule learned: **do not hide a required side effect in the right operand of && or ||**, because it runs only conditionally. When the log entry is mandatory, call the function explicitly, e.g. `ok = has_card() ? verify_card() : 0;`, then combine the results.

2.7 Alternative concept: nested if, or the non-short-circuit operators

Version	Slow calls for n passengers	Measured (n = 1000)	Time class
&& / || with guard first (V1, V2)	between 0 and n; about n x p_{card} x (1 - $p_{S\ and\ E}$) for V1, n x p_{card} for V2	78 and 89	O(n) worst case, small constant
Nested if (V5)	same as V1 (the same branches are taken)	78	O(n) worst case, same as V1
Slow call written first, or & and | (V3, V4)	exactly n	1000	Theta(n), always the maximum

Space is O(1) for all versions (only counters), except the audit log, which grows with the number of calls. Nested if gives no speed advantage over a well-ordered && / ||; it is preferable only when each branch needs a different action (for example a different message). The non-short-circuit operators remove the saving completely and should not be used on conditions.

Evaluation flow of V1 / V5 (diagram)



Both No branches of the first two tests join the card check. The slow function is reached only through the card-holder test, which is why V1 and V2 call it so rarely.

Pseudocode

```
FOR each passenger p
granted <- FALSE
    IF p.senior AND p.eligible THEN granted <- TRUE          (cheap, tested first)
    IF NOT granted AND p.hasCard THEN                          (cheap guard)
        IF verify_card(p) THEN granted <- TRUE                 (slow call, only when needed)
    PRINT granted
```

Source code

```
#include <stdio.h>
#include <stdlib.h> /* Q2: short-circuit experiments for the
railway concession rule.
   Rule: concession = (senior AND eligible class) OR (disability card holder AND card verifies). */
#define N 1000

static unsigned long long seed = 12345;
static int rnd(int mod) { seed = seed * 6364136223846793005ULL + 1442695040888963407ULL; return (int)((seed >> 33) % (unsigned)mod); }

int senior_[N], elig_[N], card_[N]; unsigned
long slow_calls, s_calls, e_calls; int
audit_log[N]; int audit_len;
int cur;                                  /* index of passenger being processed */

int is_senior(void)   { s_calls++; return senior_[cur]; }
int is_eligible(void) { e_calls++; return elig_[cur]; } int
has_card(void)    { return card_[cur]; }
int verify_card(void) { slow_calls++; audit_log[audit_len++] = cur; return 1; } /* slow server call, card is genuine */

void generate(int pc_senior, int pc_elig, int pc_card)
{
    seed = 12345;     for (int i = 0; i <
N; i++) {         senior_[i] = rnd(100) <
pc_senior;         elig_[i]   = rnd(100) <
pc_elig;         card_[i]   = rnd(100) <
pc_card;
    }
}

typedef int (*Rule)(void); int v1(void) { return (is_senior() && is_eligible()) ||
(has_card() && verify_card()); } int v2(void) { return (has_card() && verify_card()) ||
(is_senior() && is_eligible()); } int v3(void) { return (verify_card() && has_card()) ||
(is_senior() && is_eligible()); } int v4(void) { return (is_senior() & is_eligible()) |
(has_card() & verify_card()); } int v5(void)
{   /* nested-if equivalent of v1 */     int g = 0;     if
(is_senior()) { if (is_eligible()) g = 1; }     if (!g) { if
(has_card()) { if (verify_card()) g = 1; } }     return g;
}
int v6(void) { return (is_eligible() && is_senior()) || (has_card() && verify_card()); } /* cheap order swapped */
void run(const char *name, Rule r, int show_cheap)
{
    slow_calls = s_calls = e_calls = 0; audit_len = 0;     int granted = 0;     for (cur = 0; cur < N;
cur++) granted += r();     printf("%-34s slow calls=%4lu  granted=%3d  audit entries=%4d", name, slow_calls,
granted, audit_len);     if (show_cheap) printf("  senior/elig evals=%lu/%lu (sum %lu)", s_calls, e_calls,
s_calls + e_calls);     printf("\n"); }

int main(void)
{
    printf("== Part 1: 1000 passengers (25%% senior, 40%% eligible class, 8%% card holders) ==\n");     generate(25,
40, 8);     int ns = 0, ne = 0, nc = 0, nse = 0;     for (int i = 0; i < N; i++) { ns += senior_[i]; ne += elig_[i]; nc
+= card_[i]; nse += senior_[i] && elig_[i]; }     printf("actual: senior=%d eligible=%d card=%d senior&&eligible=%d\n",
ns, ne, nc, nse);     run("V1 (S&&E)||(C&&verify)",     v1, 1);     run("V2 (C&&verify)||(S&&E)",     v2, 1);
run("V3 (verify&&C)||(S&&E)",     v3, 1);     run("V4 (S&E)|(C&verify) bitwise", v4, 1);     run("V5 nested if (same as
V1)",  v5, 1);     run("V6 (E&&S)||(C&&verify)",     v6, 1);

    printf("\n== Part 2: card-holder prevalence sweep (slow calls out of 1000) ==\n");     printf("%-8s %-6s %-6s %-6s %-6s\n",
"card%", "V1", "V2", "V3", "V4");     int sweep[] = {1, 8, 25, 50, 90};     for (int k = 0; k < 5; k++) {         generate(25, 40,
sweep[k]);         unsigned long c[4]; Rule rs[4] = {v1, v2, v3, v4};         for (int j = 0; j < 4; j++) { slow_calls = 0;
audit_len = 0; for (cur = 0; cur < N; cur++) rs[j](); c[j] = slow_calls; }         printf("%-8d %-6lu %-6lu %-6lu %-6lu\n",
sweep[k], c[0], c[1], c[2], c[3]);     }

    printf("\n== Part 3: when is the 2nd operand NOT evaluated? ==\n");     int a = 0, b =
0, c = 0, d = 0, e = 0, f = 0, g = 0, h = 0;     int r1 = 0 && (a = 1);   /* && : left
false -> right skipped */     int r2 = 1 && (b = 1);   /* && : left true  -> right
evaluated */     int r3 = 1 || (c = 1);   /* || : left true  -> right skipped */     int r4
= 0 || (d = 1);   /* || : left false -> right evaluated */     int r5 = 0 & (e = 1);    /*
&  : always evaluates both */     int r6 = 1 | (f = 1);    /* |  : always evaluates both */
int r7 = 2 & 1;          /* bitwise: 10 & 01 = 0 although both operands are 'true' */
int r8 = 2 && 1;         /* logical: 1 */
    (void)g; (void)h;
    printf("0 && (a=1): result=%d a=%d\n", r1, a);
printf("1 && (b=1): result=%d b=%d\n", r2, b);     printf("1
|| (c=1): result=%d c=%d\n", r3, c);     printf("0 || (d=1):
result=%d d=%d\n", r4, d);     printf("0 &  (e=1): result=%d
e=%d\n", r5, e);     printf("1 |  (f=1): result=%d f=%d\n",
r6, f);     printf("2 & 1 = %d   but   2 && 1 = %d\n", r7,
r8);     return 0;
}
```

Q3 - Device-Status Register with Bitwise Operators

Smart-home controller. 6 marks, Bloom K4 (Analyse). Includes Complete the Code.

3.1 Problem analysis

One **unsigned char** holds eight device states, bit *n* for device *n* (LIGHT = bit 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4). A mask with only bit *dev* set is `1u << dev`. Each operation uses one property of a bitwise operator:

Operation	Expression	Why it works
Switch on	reg | (1u << dev)	x OR 1 = 1 forces the bit to 1; x OR 0 = x leaves the other bits unchanged
Switch off	reg & ~(1u << dev)	~mask has a 0 only at bit dev; x AND 0 = 0 clears it, x AND 1 = x keeps the others
Toggle	reg ^ (1u << dev)	x XOR 1 flips x; x XOR 0 = x
Test	(reg & (1u << dev)) != 0	AND with the mask keeps only bit dev; the result is non-zero exactly when it is 1
Print in binary	((reg >> bit) & 1u)	shift bit to position 0, then mask it
Count devices ON	while (r) { r &= r - 1; n++; }	r & (r - 1) clears the lowest set bit, so the loop runs once per ON device

3.2 Complete the Code

TODO 1 to 4 were completed using only bitwise operators; the rest of the program is unchanged.

```
#include <stdio.h>
/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);          /* TODO 1: set bit dev */
}
unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);         /* TODO 2: clear bit dev */
}
unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);          /* TODO 3: flip bit dev */
}
int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;   /* TODO 4: test bit dev */
}
void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
putchar('\n');
}
int main(void)
{
    unsigned char reg = 0;
reg = switch_on(reg, LIGHT);
reg = switch_on(reg, AC);
show(reg);
    reg = toggle(reg, FAN);
reg = switch_off(reg, LIGHT);
show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
is_on(reg, PUMP) ? "ON" : "OFF");     return 0; }
```

Prediction: register value after each statement of main()

Statement	Register (binary)	Decimal	Printed by show()
unsigned char reg = 0;	00000000	0	-
reg = switch_on(reg, LIGHT);	00000001	1	-
reg = switch_on(reg, AC);	00000101	5	00000101
reg = toggle(reg, FAN);	00000111	7	-

reg = switch_off(reg, LIGHT);	00000110	6	00000110
is_on(reg, AC) (bit 2)	bit 2 = 1	-	AC is ON
is_on(reg, PUMP) (bit 4)	bit 4 = 0	-	PUMP is OFF

Observed output (program run)	Expected output given in the assignment	Match
00000101 00000110 AC is ON, PUMP is OFF	00000101 00000110 AC is ON, PUMP is OFF	Identical

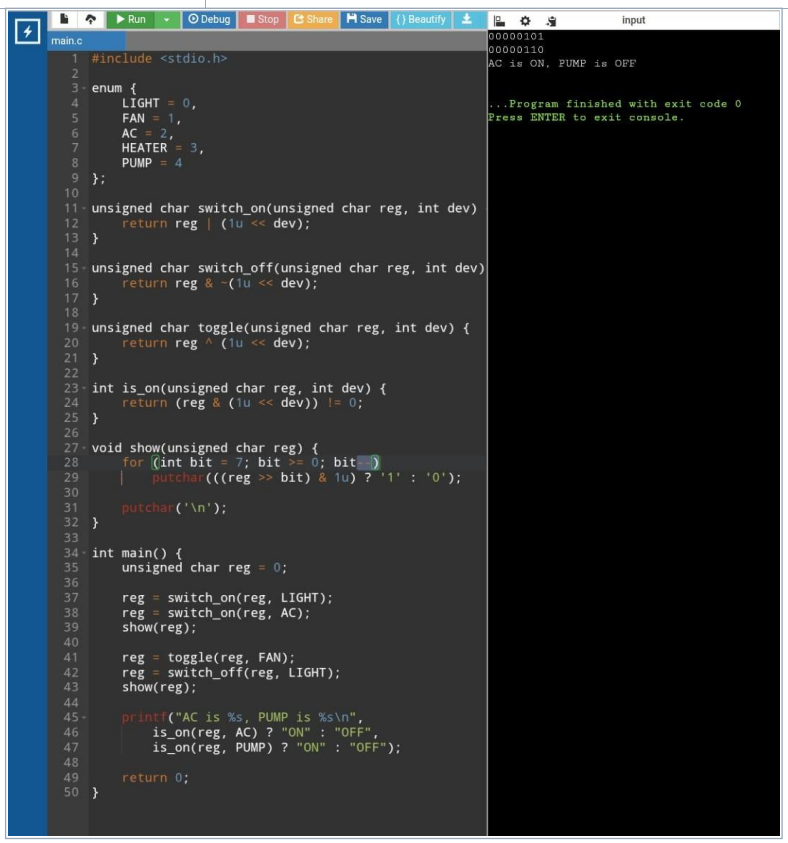


Figure 3 - Student's execution of the completed program: 00000101, 00000110, "AC is ON, PUMP is OFF".

3.3 My own predicted sequence and register trace

A second sequence was designed to include the boundary device (bit 7), an operation that changes nothing (clearing a bit that is already 0) and toggles. Each predicted value was written down first; the program `q3_trace.c` then compared them.

```
== Part 1: predicted vs observed register after each statement ==
statement              pred.bin  dec  obs.bin   dec  match
switch_on(HEATER)      00001000 8    00001000 8    yes
switch_on(PUMP)        00011000 24   00011000 24   yes
switch_on(FAN)         00011010 26   00011010 26   yes
toggle(HEATER)         00010010 18   00010010 18   yes
toggle(LIGHT)          00010011 19   00010011 19   yes
switch_off(PUMP)       00000011 3    00000011 3    yes
switch_off(AC)         00000011 3    00000011 3    yes
switch_on(device 7)    10000011 131  10000011 131  yes devices ON
in final register = 3 (loop iterations = 3)
```

All eight predictions matched. Note that `switch_off(AC)` on a bit that is already 0 leaves the register unchanged (idempotent), and that three devices are ON at the end (LIGHT, FAN, device 7). The Kernighan counting loop ran exactly 3 iterations for 3 ON devices.

3.4 Inquiry: precedence and signedness

reg & 1 << dev versus (reg & 1) << dev

In C, **<< has higher precedence than &**. So `reg & 1 << dev` means `reg & (1 << dev)`: it tests bit *dev*, which is what is wanted. In `(reg & 1) << dev` the brackets force the AND first: it extracts bit 0 only (the LIGHT bit) and then moves that result to position *dev*. It does not look at bit *dev* at all.

```
reg=00000100 (AC on): reg & 1 << 2 = 4 ; (reg & 1) << 2 = 0
reg=00000001 (LIGHT on, AC off): reg & 1 << 2 = 0 ; (reg & 1) << 2 = 4
```

With only AC on, the correct form gives 4 (non-zero, AC is on) but the wrong form gives 0 (AC looks off). With only LIGHT on, the wrong form gives 4 and falsely reports AC as on. A related trap: `reg & mask == 0` parses as `reg & (mask == 0)` because == binds tighter than &, so the test must be written `(reg & mask) != 0`.

What goes wrong with signed char when bit 7 is set

```
signed char : value=-128  s>>7=-1  (s>>7)==1:0  s==0x80:0  s<0:1
unsigned    : value=128  u>>7=1  (u>>7)==1:1  u==0x80:1  u<0:0 bit
test (s & (1<<7))!=0 -> 1 ; ~s as int = 127 ; ~u as int = -129
(unsigned char)~u = 127
```

- Setting bit 7 makes a signed char **negative** (observed value -128, and s < 0 is true). Converting 128 into signed char is implementation-defined.
- When a char is used in an expression it is promoted to int with **sign extension**. The register compares as -128, so `s == 0x80` is false (0x80 is 128) while the unsigned version gives true.
- Right shift of a negative value is implementation-defined; GCC does an **arithmetic shift**, filling with 1s, so `s >> 7` is -1, not 1, and `(s >> 7) == 1` is false.
- Printing the register as a number shows -128 instead of 128. The mask test `(s & (1 << 7)) != 0` still works, which hides the problem until another operation is used.
- Even for unsigned char, integer promotion applies: `~u` is the int -129, not an 8-bit value. The functions return unsigned char, so the result is truncated back to 8 bits, but intermediate results printed or compared must be cast: `(unsigned char)~u` = 127. **Rule: keep the register unsigned (unsigned char or uint8_t) and use 1u masks.**

3.5 Alternative concept: char state[8] (one byte per device)

```
void on(char s[8], int d)      { s[d] = 1; } void
off(char s[8], int d)     { s[d] = 0; } void
toggle(char s[8], int d) { s[d] ^= 1; } int
is_on(char s[8], int d)  { return s[d]; }
int  count(char s[8])          { int n = 0; for (int i = 0; i < 8; i++) n += s[i]; return n; }
```

```
sizeof(unsigned char)=1 byte, sizeof(HomeArr)=8 bytes
1,000,000 homes: bit-register = 1000000 bytes, array = 8000000 bytes
count devices ON, 1,000,000 homes x 20 repeats (totals must agree: 79939780 79939780 79939780)
array scan     : 160000000 loop iterations per pass-set, 0.085 s bit (Kernighan): 79939780 loop
iterations per pass-set, 0.198 s bit + 256-table: 20000000 lookups per pass-set,           0.008
s
```

Measure	One bit per device (unsigned char)	Array char state[8]
Time: set, clear, toggle, test	O(1): one shift plus one bitwise operation	O(1): one indexed access
Space per home	1 byte	8 bytes (sizeof = 8, measured)
Space for 1,000,000 homes	1,000,000 bytes (about 1.0 MB)	8,000,000 bytes (about 8.0 MB); 7 MB more
Count devices ON, per home	Kernighan loop: one iteration per ON bit; average 4.00 over all 256 register values (measured total 1024); a 256-entry table gives 1 lookup	always 8 iterations (2048 over all 256 states)
Count, 1,000,000 homes x 20 passes	79,939,780 iterations, 0.198 s (table version: 20,000,000 lookups, 0.008 s)	160,000,000 iterations, 0.085 s
Totals agree?	yes (79,939,780)	yes (79,939,780)

Analysis. Per-device operations are O(1) in both designs, and counting the ON devices of one home is O(8) = O(1) in both; over H homes counting is O(H). The bit register uses one eighth of the memory, which matters on a controller with a few bytes of RAM and when a million states are stored or sent over a network. The array is easier to read and extends beyond 8 devices without changing the type. Counting is the interesting case: the Kernighan loop does half the iterations of the array scan (4.0 on average versus 8) but was *slower* in wall-clock time here (0.198 s versus 0.085 s at -O2), probably because its loop length depends on the data and the branch is hard to predict, whereas the array loop has a fixed length. The 256-entry lookup table (256 extra bytes) was fastest by a wide margin (0.008 s). Timings are machine-specific; the iteration counts are exact. The bit register also allows whole-register operations in one instruction (switch everything off with reg = 0, compare two states with ^).

Source code

```
#include <stdio.h>
#include <assert.h>
/* Q3: register trace with predictions, popcount, precedence and signedness experiments */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4, DEV7 = 7 }; unsigned char
switch_on(unsigned char r, int d)  { return r | (1u << d); } unsigned char
switch_off(unsigned char r, int d) { return r & ~(1u << d); } unsigned char toggle(unsigned
```

```
char r, int d)     { return r ^ (1u << d); } int is_on(unsigned char r, int d)
{ return (r & (1u << d)) != 0; }
void bin(unsigned char r) { for (int b = 7; b >= 0; b--) putchar(((r >> b) & 1u) ? '1' : '0'); }

int count_on(unsigned char r, int *iters)   /* Kernighan: each pass clears the lowest set bit */
{
    int n = 0; *iters = 0;     while (r) { r &=
r - 1; n++; (*iters)++; }     return n; }

int main(void)
{
    printf("== Part 1: predicted vs observed register after each statement ==\n");
struct { const char *stmt; int op, dev; unsigned char predicted; } steps[] = {
        {"switch_on(HEATER)",  0, HEATER, 0x08}, {"switch_on(PUMP)",    0, PUMP,   0x18},
        {"switch_on(FAN)",     0, FAN,    0x1A}, {"toggle(HEATER)",     2, HEATER, 0x12},
        {"toggle(LIGHT)",      2, LIGHT,  0x13}, {"switch_off(PUMP)",   1, PUMP,   0x03},
{"switch_off(AC)",     1, AC,     0x03}, {"switch_on(device 7)",0, DEV7,   0x83},
    };
    unsigned char reg = 0;     printf("%-22s %-9s %-4s %-9s %-4s %s\n", "statement", "pred.bin", "dec",
"obs.bin", "dec", "match");     for (int i = 0; i < 8; i++) {         unsigned char before = reg;
reg = steps[i].op == 0 ? switch_on(reg, steps[i].dev)
            : steps[i].op == 1 ? switch_off(reg, steps[i].dev) : toggle(reg, steps[i].dev);
        (void)before;
        printf("%-22s ", steps[i].stmt); bin(steps[i].predicted); printf(" %-4d ", steps[i].predicted);
bin(reg); printf(" %-4d %s\n", reg, reg == steps[i].predicted ? "yes" : "NO");
    }
    int it; int n = count_on(reg, &it);
    printf("devices ON in final register = %d (loop iterations = %d)\n", n, it);

    printf("\n== Part 2: reg & 1 << dev  versus  (reg & 1) << dev ==\n");     unsigned char t1 = 0x04, t2 = 0x01; int dev =
2;     printf("reg=00000100 (AC on): reg & 1 << 2 = %d ; (reg & 1) << 2 = %d\n", t1 & 1 << dev, (t1 & 1) << dev);
printf("reg=00000001 (LIGHT on, AC off): reg & 1 << 2 = %d ; (reg & 1) << 2 = %d\n", t2 & 1 << dev, (t2 & 1) << dev);

    printf("\n== Part 3: signed char with bit 7 set ==\n");     signed char s = 0; s = s | (1 << 7);     unsigned char u = 0; u
= u | (1 << 7);     printf("signed char : value=%d  s>>7=%d  (s>>7)==1:%d  s==0x80:%d  s<0:%d\n", s, s >> 7, (s >> 7) == 1, s ==
0x80, s < 0);     printf("unsigned    : value=%d  u>>7=%d  (u>>7)==1:%d  u==0x80:%d  u<0:%d\n", u, u >> 7, (u >> 7) == 1, u ==
0x80, u < 0);     printf("bit test (s & (1<<7))!=0 -> %d ; ~s as int = %d ; ~u as int = %d\n", (s & (1 << 7)) != 0, ~s, ~u);
printf("(unsigned char)~u = %d\n", (unsigned char)~u);

    printf("\n== Part 4: count-ON cost over all 256 register values ==\n");     long tot_k = 0;     for (int v = 0; v < 256; v++) { int
i2; count_on((unsigned char)v, &i2); tot_k += i2; }     printf("Kernighan total iterations = %ld (average %.2f); array scan = 8 per
register (%d total)\n", tot_k, tot_k / 256.0, 256 * 8);     return 0; }
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
/* Q3 alternative: one byte per device (char state[8]) versus one bit per device (unsigned char) */
#define HOMES 1000000
typedef struct { char state[8]; } HomeArr;

int main(void)
{
    printf("sizeof(unsigned char)=%zu byte, sizeof(HomeArr)=%zu bytes\n", sizeof(unsigned char),
sizeof(HomeArr));     printf("1,000,000 homes: bit-register = %zu bytes, array = %zu bytes\n",
sizeof(unsigned char) * (size_t)HOMES, sizeof(HomeArr) * (size_t)HOMES);

    unsigned char *regs = malloc(HOMES);
    HomeArr *arr = malloc(sizeof(HomeArr) * HOMES);     unsigned int s
= 2463534242u;     for (int i = 0; i < HOMES; i++) {         s ^= s <<
13; s ^= s >> 17; s ^= s << 5;         regs[i] = (unsigned char)s;
for (int d = 0; d < 8; d++) arr[i].state[d] = (regs[i] >> d) & 1;
    }
    unsigned char popc[256];
    for (int v = 0; v < 256; v++) { int c = 0; for (int b = 0; b < 8; b++) c += (v >> b) & 1; popc[v] = (unsigned char)c; }

    long total_a = 0, total_k = 0, total_t = 0, it_a = 0, it_k = 0;     clock_t t0 = clock();     for (int r = 0; r < 20; r++) for
(int i = 0; i < HOMES; i++) { for (int d = 0; d < 8; d++) { total_a += arr[i].state[d]; it_a++; } }     clock_t t1 = clock();     for
(int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) { unsigned char v = regs[i]; while (v) { v &= v - 1; total_k++; it_k++; } }
clock_t t2 = clock();     for (int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) total_t += popc[regs[i]];     clock_t t3 =
clock();     printf("count devices ON, 1,000,000 homes x 20 repeats (totals must agree: %ld %ld %ld)\n", total_a, total_k, total_t);
printf("array scan     : %ld loop iterations per pass-set, %.3f s\n", it_a, (double)(t1 - t0) / CLOCKS_PER_SEC);     printf("bit
(Kernighan): %ld loop iterations per pass-set, %.3f s\n", it_k, (double)(t2 - t1) / CLOCKS_PER_SEC);     printf("bit + 256-table: %d
lookups per pass-set,           %.3f s\n", 20 * HOMES, (double)(t3 - t2) / CLOCKS_PER_SEC);     free(regs); free(arr);     return 0;
}
```

Q4 - else-if Ladder or switch?

University grade and scholarship classifier. 6 marks, Bloom K4 (Analyse). Includes Complete the Code.

4.1 Problem and implementations

Grade rules: 90-100 S, 80-89 A, 70-79 B, 60-69 C, 50-59 D, below 50 F; marks outside 0-100 give `'?'`. Three implementations were written: an else-if ladder, a switch on `marks / 10` (TODO 1), and the lookup table from the alternative concept.

else-if ladder (student's program, Figure 4)

```
char grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
else if (marks >= 90) return 'S';     else if
(marks >= 80) return 'A';     else if (marks
>= 70) return 'B';     else if (marks >= 60)
return 'C';     else if (marks >= 50) return
'D';     else return 'F'; }
```

switch on marks / 10 (TODO 1 completed)

```
#include <stdio.h>
/* Grade rules: 90-100 S, 80-89 A, 70-79 B, 60-69 C, 50-59 D, below 50 F, invalid '?' */
char grade(int marks)
{
    if (marks < 0 || marks > 100) return '?';
switch (marks / 10) {              /* TODO 1 */
case 10:
        case 9: return 'S';
case 8: return 'A';
case 7: return 'B';
case 6: return 'C';
case 5: return 'D';
default: return 'F';
    }
}
int main(void)
{
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++) printf("%d:%c ", marks[i], grade(marks[i]));
printf("\n");     return 0; }
```

Observed output (both versions)	Expected output given in the assignment	Match
100:S 95:S 83:A 50:D 49:F -5:?	100:S 95:S 83:A 50:D 49:F -5:?	Identical

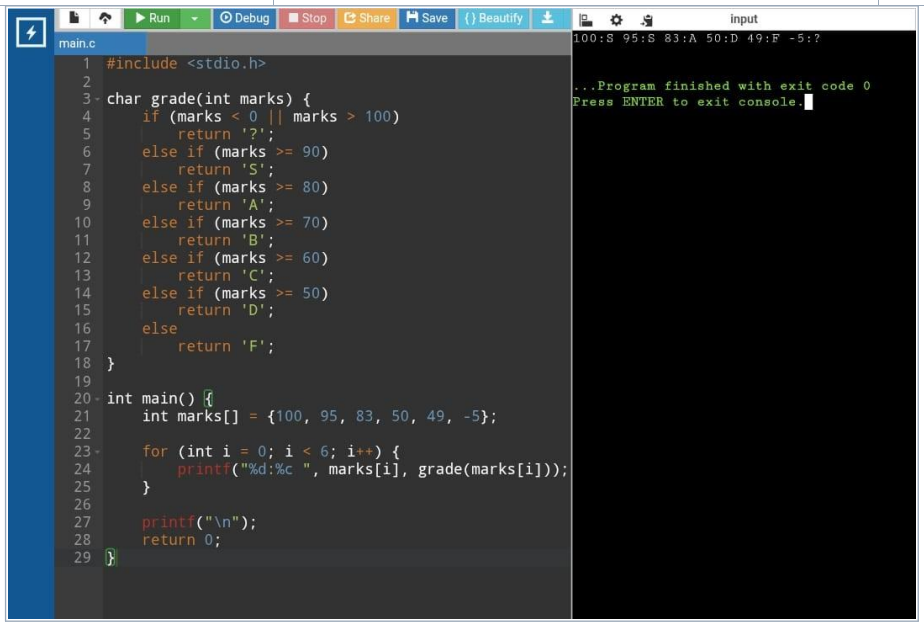


Figure 4 - Student's execution of the else-if ladder version.

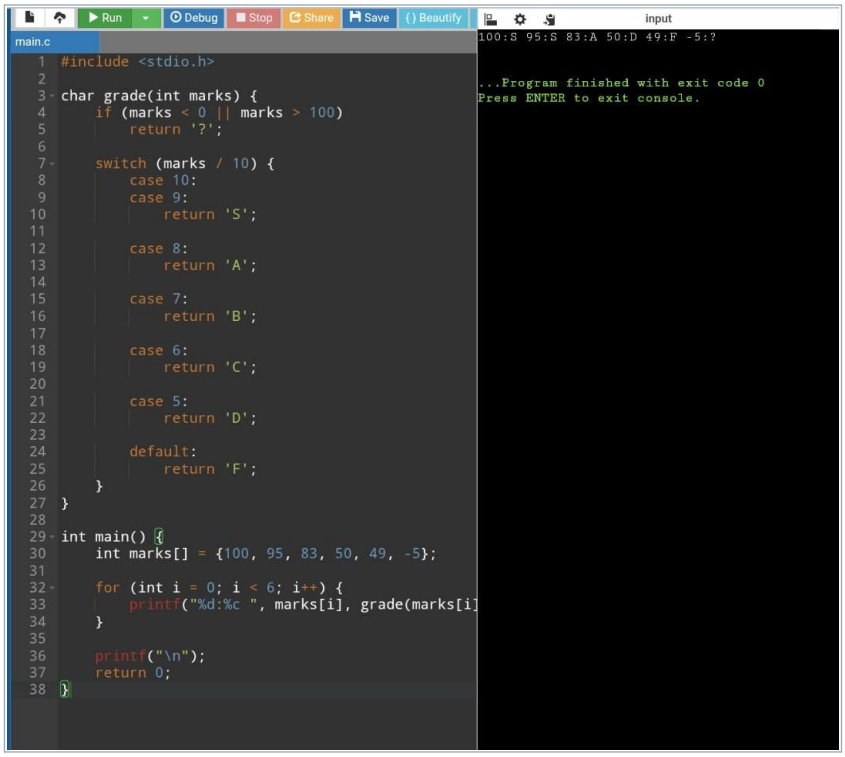


Figure 5 - Student's execution of the switch version (TODO 1 completed with case 10 and case 9 combined).

4.2 Which inputs are handled correctly and incorrectly

The six required inputs were tested on all three versions, with and without the validation line `if (marks < 0 || marks > 100) return '?';` (columns "no guard").

marks	Expected	Ladder	switch	Table	Ladder, no guard	switch, no guard	Table, no guard
100	S	S	S	S	S	S	S
90	S	S	S	S	S	S	S
89	A	A	A	A	A	A	A
0	F	F	F	F	F	F	F
-1	?	?	?	?	**F (wrong)**	**F (wrong)**	**F (wrong)**
101	?	?	?	?	**S (wrong)**	**S (wrong)**	**S (wrong)**

- With the guard all three versions are correct for all six inputs.
- Without the guard, -1 gives F and 101 gives S in **every** version. For the switch the reason is that integer division truncates toward zero, so -1 / 10 = 0 (case 0, default F) and 101 / 10 = 10 (case 10, S). The validation line is therefore essential, not optional. For the table, a value such as -10 or 110 would index outside the array (undefined behaviour), so the guard also protects memory.
- The switch is correct for 100 only because `case 10` is combined with `case 9`. If that label were forgotten, 100 / 10 = 10 would fall to the default and 100 would wrongly become F (by reasoning from the code).

4.3 Effect of a missing break (or return)

The switch was rewritten with `break` (program `q4_fallthrough.c`) so that one break could be removed at a time. A missing return behaves identically in the submitted version, because both end the case.

```
BUG=0: 100:S 95:S 83:A 75:B 65:C 50:D 49:F -5:?
BUG=1: 100:A 95:A 83:A 75:B 65:C 50:D 49:F -5:?
BUG=2: 100:S 95:S 83:B 75:B 65:C 50:D 49:F -5:?
BUG=3: 100:S 95:S 83:A 75:B 65:C 50:F 49:F -5:?
```

Variant	What was removed	Changed output	Why
BUG=0	nothing	none (reference)	-
BUG=1	break after case 9	100:A and 95:A (should be S)	Execution falls from case 9 into case 8 and overwrites S with A

BUG=2	break after case 8	83:B (should be A)	Falls from case 8 into case 7 and the grade becomes B
BUG=3	break after case 5	50:F (should be D)	Falls from case 5 into default and the grade becomes F

A missing break does not cause an error: the program compiles and runs, and the damage depends on the input. Only grades whose case precedes another case are affected, so a small test set can miss it. GCC reports it with `-Wextra` (-Wimplicit-fallthrough): "this statement may fall through". The deliberate empty `case 10: case 9:` is not flagged because it has no statements.

4.4 Why switch cannot directly test a range such as 75-84

- A case label must be an **integer constant expression** and is compared by **equality**. Relational tests such as `marks >= 75 && marks <= 84` are not allowed in a label.
- A range would need ten labels (case 75: ... case 84:), or the GCC extension `case 75 ... 84:`, which is not standard C.
- The expression `marks / 10` collapses 70-79 and 80-89 into buckets 7 and 8, so 75-84 straddles two buckets and cannot be expressed. Using `marks / 5` (75-79 gives 15, 80-84 gives 16) would work only because 75 and 85 are multiples of 5, and every other label would have to be rewritten.

4.5 Which version is easier to change when the grade boundaries change?

Version	Cost of changing a boundary (for example S from 90 to 85)
else-if ladder	Change one constant (90 to 85). Order of the tests must stay descending. Easiest.
switch on marks / 10	Boundaries must be multiples of the divisor. 85 is not, so the divisor becomes 5 and all case labels are rewritten.
lookup table	Boundary on a multiple of 10: edit one entry. Otherwise a larger table (up to 101 entries) or a different divisor. A data-driven variant (arrays of thresholds and letters scanned by a loop) is a one-line change.

4.6 Test set that exposes every boundary error

Boundary errors typically come from writing > for >= (or the reverse). To check the test set objectively, seven faulty versions (mutants) of the ladder were created: five with > at one grade threshold, and two with an off-by-one in the validation. The test set is {-1, 0, 49, 50, 59, 60, 69, 70, 79, 80, 89, 90, 99, 100, 101}.

```
mutant >90 (not >=)      killed by: 90    (1 tests)
mutant >80               killed by: 80    (1 tests)
mutant >70               killed by: 70    (1 tests)
mutant >60               killed by: 60    (1 tests)
mutant >50               killed by: 50    (1 tests)
mutant m<=0 invalid      killed by: 0    (1 tests)
mutant m>=100 invalid    killed by: 100    (1 tests)
```

Finding: every faulty version is exposed by the *exact* boundary value and by no other test. The neighbours (89, 99, 59, ...) do not detect a >= written as >. A test set built only from "typical" values such as 95, 83, 50 would detect none of the ladder errors except at 50. All 15 values passed on the three correct versions (failures = 0), and the three versions agreed on every integer from -20 to 120.

4.7 Lookup-table alternative and complexity

```
char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'}; char
grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
return grades[marks / 10]; }
```

```
marks  100 -> 3 comparisons
marks   90 -> 3 comparisons
marks   89 -> 4 comparisons
marks    0 -> 7 comparisons
marks   -1 -> 1 comparisons
marks  101 -> 2 comparisons
all valid marks 0..100: total=603, average=5.97 comparisons
worst case = 7 comparisons (grade F) three
implementations agree on -20..120: yes
```

Measure	else-if ladder	switch (marks / 10)	Lookup table
Comparisons per grade	up to k: 3 for S, 7 for F (including the 2 validation tests); average 5.97 over 0-100	1 division, then a jump (the compiler chooses a jump table or a comparison tree)	1 division and 1 array read, plus the 2 validation tests

Time per grade	O(k) with k = 6 grade levels (a constant, so O(1) for fixed rules)	O(1)	O(1)
Space	none	none (code only)	11 bytes
Measured, 30,000,000 calls, -O0	0.145 s	0.120 s	0.099 s

The timing gap is small (30 million calls differ by less than 0.05 s) and machine-specific; it should not drive the decision. The comparison counts show the real difference: the ladder does more work for low grades, the table does a constant amount of work. For six fixed grades the ladder is perfectly adequate, and it is the only one of the three that handles arbitrary boundaries naturally.

4.8 Design decision: the scholarship rule

The assignment does not give the scholarship rule, so an **illustrative rule** was assumed: three income bands (low below Rs 3 lakh, middle Rs 3 to 6 lakh, high above Rs 6 lakh) and four categories. Replace the table with the real rule if it differs; the argument does not change.

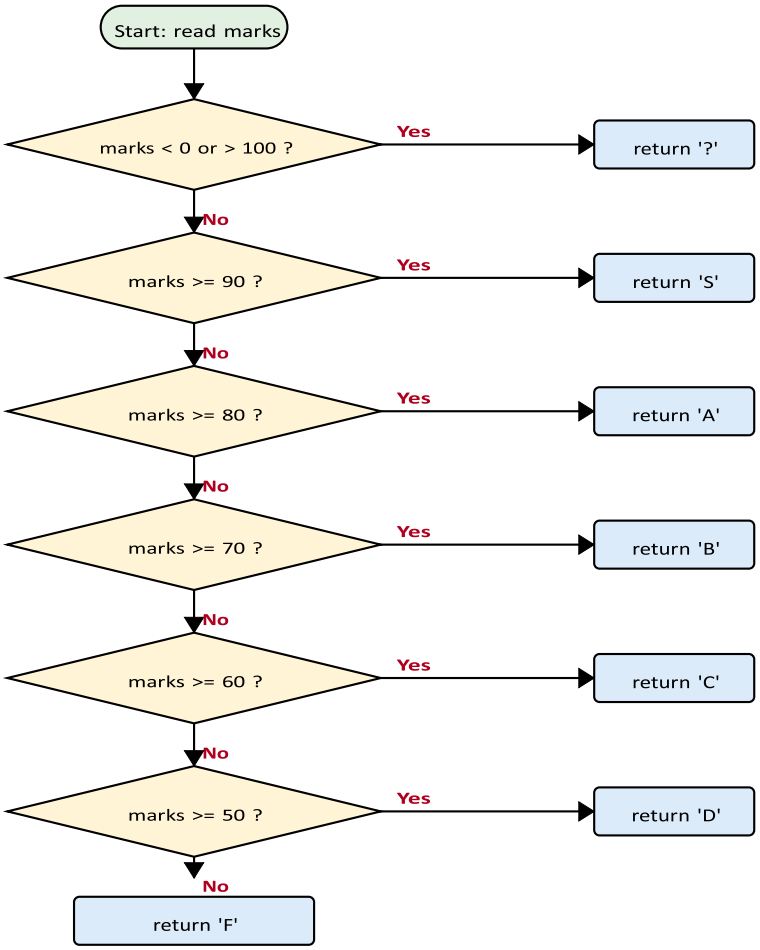
Grade	Low income	Middle income	High income
S	Full	Full	Half
A	Full	Half	Quarter
B	Half	Quarter	None
C	Quarter	None	None
D, F	None	None	None
? (invalid marks)	Rejected	Rejected	Rejected

Structure	Assessment
Nested if	Works, but the 6 x 3 rule is hidden in control flow: many branches, easy to mis-nest, every new grade or band edits several places, and the rule cannot be reviewed as a table.
switch inside an if	Moves the nesting into another statement. It reads better for the grade, but the income band still needs a second decision, and each grade case still repeats an inner if.
Lookup table (chosen)	The rule is data, not logic. One row per grade, one column per band; a reviewer can compare it directly to the policy. A new band is a new column. Lookup is O(1). Cost: a small constant table (18 entries) and an index for the grade.

Decision: a two-dimensional lookup table for grade x income band, with an if for the validation of marks and a short ladder to turn income into a band. The table cannot index income directly (income is a range, the same limitation as in 4.4), so the band is found by comparison, and the table is indexed by the band and the grade. In the experiment the table version and a fully nested-if version were compared on **42 cases** (7 grade values x 6 incomes, including the boundary incomes 29/30 and 60/61 and the invalid grade): **0 differences**.

```
cells tested=42, differences=0
grade S: low=Full     mid=Full     high=Half    grade
A: low=Full     mid=Half     high=Quarter grade B:
low=Half     mid=Quarter  high=None    grade C:
low=Quarter  mid=None     high=None    grade D:
low=None     mid=None     high=None    grade F:
low=None     mid=None     high=None
```

Flowchart of the else-if ladder



Pseudocode

```
FUNCTION grade(marks)
    IF marks < 0 OR marks > 100 THEN RETURN '?'
    index <- marks DIV 10                (10 for 100, 9 for 90-99, ..., 0 for 0-9)
RETURN GRADES[index]                  (table: F F F F F D C B A S S)

FUNCTION scholarship(grade, income)
    IF grade = '?' THEN RETURN "Rejected"
    band <- 0 IF income < 3 lakh, 1 IF income <= 6 lakh, ELSE 2
RETURN TABLE[row of grade][band]
```

Key source excerpt (full source in Q4/q4_tests.c)

```
static const char *T[6][3] = {
    /*        low        mid        high  */
    /* S */ {"Full",    "Full",    "Half"},
    /* A */ {"Full",    "Half",    "Quarter"},
    /* B */ {"Half",    "Quarter", "None"},
    /* C */ {"Quarter", "None",    "None"},
    /* D */ {"None",    "None",    "None"},
    /* F */ {"None",    "None",    "None"} };
const char *order = "SABCDF"; const char *p = strchr(order, g);
if (!p) return "Rejected";
int band = income < 30 ? 0 : income <= 60 ? 1 : 2;   /* income in units of Rs 10,000 */
return T[p - order][band];
```

Summary of Results (Q1 to Q4)

Q	Main result	Key evidence
Q1	The one-line expression charges GST on the gross amount and truncates in integer division; count = count++ + 1 is undefined behaviour.	Bill A: 1080 versus intended 1071 (+Rs 9); Bill B: 267 versus 265.07; -Wall gives -Wsequence-point. Intermediate variables: 8 operations instead of 10.

Q2	Guarding the slow call with the cheap card check reduces slow calls from 1000 to 78-89 per 1000 passengers.	V1 = 78, V2 = 89, V3 = V4 = 1000, nested if = 78. With an audit log, card-first order (V2) is required if every card holder must be logged.
Q3	One byte holds 8 devices; | sets, & ~ clears, ^ toggles, & tests. Output matches the expected output exactly.	8 of 8 predicted registers matched; reg & 1 << dev = 4 versus (reg & 1) << dev = 0; 1 MB versus 8 MB for a million homes.
Q4	Ladder and switch give identical grades for valid and invalid marks only when the range check is present; a missing break changes results silently.	15-value boundary set kills 7 of 7 mutants; each is caught only by the exact boundary value; ladder average 5.97 comparisons versus 1 table read.

Appendix A - Repository layout and build commands

Folder names follow the assignment (Q1 to Q4). Each folder holds the source, the captured text output and the screenshots. At least one commit per question is required.

```
CSA02-<RegisterNumber>-Assignment2/
  README.md                    (name, register no., GCC version, commands, short result notes)
  Q1/  q1_compare.c  q1_steps.c  q1_count.c  + output screenshots
  Q2/  q2_experiment.c  + output screenshots (single-passenger program and the 1000-passenger experiment)
  Q3/  q3_complete.c  q3_trace.c  q3_array.c  + output screenshots
  Q4/  q4_switch.c  q4_fallthrough.c  q4_tests.c  q4_bench.c  + output screenshots
```

Program	Compile	Run
Q1	gcc -Wall -Wextra -o q1_compare q1_compare.c	./q1_compare
Q1 (UB warning)	gcc -Wall -Wextra -o q1_count q1_count.c	./q1_count
Q2	gcc -Wall -Wextra -O0 -o q2_experiment q2_experiment.c	./q2_experiment
Q3	gcc -Wall -Wextra -o q3_complete q3_complete.c	./q3_complete
Q3 (trace)	gcc -Wall -Wextra -o q3_trace q3_trace.c	./q3_trace
Q3 (array)	gcc -Wall -Wextra -O2 -o q3_array q3_array.c	./q3_array
Q4	gcc -Wall -Wextra -o q4_switch q4_switch.c	./q4_switch
Q4 (break test)	gcc -DBUG=1 -o q4_fallthrough q4_fallthrough.c (BUG = 0, 1, 2 or 3)	./q4_fallthrough
Q4 (tests)	gcc -Wall -Wextra -o q4_tests q4_tests.c	./q4_tests
Q4 (timing)	gcc -O0 -o q4_bench q4_bench.c	./q4_bench

Compiler for all measured data: GCC 13.3.0 on Ubuntu 24.04. The screenshots (Figures 1 to 5) come from the student's own runs; every other count in this report was produced by the programs listed above.

Question 5: Count the Work Done by a Loop (Lucky-Draw Ticket Verification)

Student name	Swamisetti Uma Sri
Register number	1965260130
GitHub repository	CSA0222<1965260130>Assessment 2
Link	https://github.com/swamisettiumasri-debug/CSA0222_swamisetti-uma-sri-1965260130-/tree/main/ASSESSMENT%202
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compile command	`gcc -Wall -Wextra -O2 -o q5_complete q5_complete.c`
Marks / Bloom level	7 marks / K4 – Analyse

1. Problem and design starting point

Prize tickets are the prime numbers. The first version tests every divisor from 2 to n−1. The organisers will verify tickets up to 1,000,003, so the number of divisions is the quantity that must be measured and reduced. Operation counts (divisions) are the primary evidence in this report; every number below was produced by the programs in the appendix.

1.1 From the principle to a loop condition

Principle given: *if n has a divisor greater than √n, it must also have a divisor smaller than √n*. Write a composite $n = a \times b$ with $a \le b$. Then $a \times a \le a \times b = n$, so the smaller factor always satisfies $a^2 \le n$. Therefore, if no i with $i^2 \le n$ divides n, no divisor exists at all and n is prime.

Question	Answer derived
Loop condition without calling sqrt()	`i * i <= n` (overflow-safe equivalent: `i <= n / i`, because for positive integers $i \le \lfloor n/i \rfloor \Leftrightarrow i^2 \le n$)
Where break / return stops the loop early	At the **first** i with `n % i == 0`: return 0 (composite) or set prime = 0 and break. If the loop ends normally, no divisor was found: return 1.
How the divisions are counted	A counter `checks++` placed immediately before each `n % i` test; it is reset to 0 at the start of every call.

1.2 Predictions before execution

For a prime n the improved loop tests i = 2 … $\lfloor \sqrt{n} \rfloor$, i.e. $\lfloor \sqrt{n} \rfloor - 1$ divisions; the original loop tests $n - 2$ divisions. An even n stops at the very first division (i = 2).

n	Type	Predicted (i*i <= n)	Predicted (i < n)
97	prime	$\lfloor 9.85 \rfloor - 1 = 8$	$97 - 2 = 95$
100	even	1	1
7919	prime	$\lfloor 88.99 \rfloor - 1 = 87$	$7919 - 2 = 7917$
1000003	prime	$\lfloor 1000.0015 \rfloor - 1 = 999$	$1000003 - 2 = 1000001$

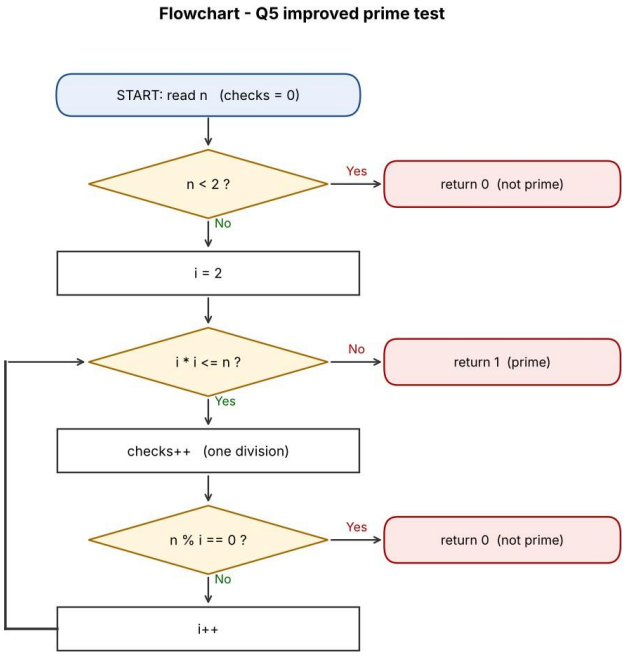


*Figure 1: flowchart of the improved prime test (condition i*i <= n, early return)*

2. Complete the Code (TODO 1)

Only the body marked TODO 1 was replaced; the rest of the program is unchanged.

```
#include <stdio.h>
unsigned long checks = 0; /* number of divisions performed by the last call */
/* Returns 1 if n is prime, otherwise 0. */
int is_prime(unsigned long n)
{
checks = 0;
if (n < 2) return 0;
for (unsigned long i = 2; i * i <= n; i++) {
    checks++;
    if (n % i == 0) return 0;
}
return 1;
}
int main(void)
{
unsigned long tickets[] = {97, 100, 7919, 1000003};
for (int k = 0; k < 4; k++) {
int p = is_prime(tickets[k]);
printf("%8lu %-9s divisions = %lu\n", tickets[k],
p ? "prime" : "not prime", checks);
}
return 0;
}
```

Logic. The loop runs while `i * i <= n`. *checks* is incremented before every division, so it equals the number of `%` operations. As soon as `n % i == 0` the function returns 0; if the loop condition fails first, no divisor $\leq \sqrt{n}$ exists and 1 is returned. Because the three-part `for` header holds the whole loop control, there is no separate flag.

2.1 Expected versus observed output (gcc 13.3.0)

Ticket	Expected output	Observed output	Match
97	prime, divisions = 8	prime, divisions = 8	yes
100	not prime, divisions = 1	not prime, divisions = 1	yes
7919	prime, divisions = 87	prime, divisions = 87	yes
1000003	prime, divisions = 999	prime, divisions = 999	yes

```
      97 prime      divisions = 8
     100 not prime divisions = 1
```

```
   7919 prime      divisions = 87
1000003 prime      divisions = 999
```

2.2 Effect of changing the condition to i < n

Using the same program with the condition replaced by `i < n` (file q5_complete_i_lt_n.c):

```
     97 prime      divisions = 95
    100 not prime  divisions = 1
   7919 prime      divisions = 7917
1000003 prime      divisions = 1000001
```

The answers are identical (prime / not prime) but the work for the prime tickets grows from 8 → 95, 87 → 7,917 and 999 → 1,000,001. For the even ticket 100 nothing changes (1 division), because the early return fires at i = 2 in both versions. This shows that the *result* does not depend on the boundary, while the *cost* does.

3. Three loop forms and the division counts

The improved test was written three times with the same condition: `for`, `while` and `do-while` (source in Appendix A, function names prime_for / prime_while / prime_dowhile). Division counts for eight tickets, including a large prime (1000003), the largest prime below 10^6 (999983), a large composite and the special cases 2 and 3:

n	Result	2..n−1 (naive)	for	while	do-while
2	prime	0	0	0	0
3	prime	1	0	0	0
97	prime	95	8	8	8
100	composite	1	1	1	1
7919	prime	7,917	87	87	87
999983	prime	999,981	998	998	998
1000001 (=101×9901)	composite	100	100	100	100
1000003	prime	1,000,001	999	999	999

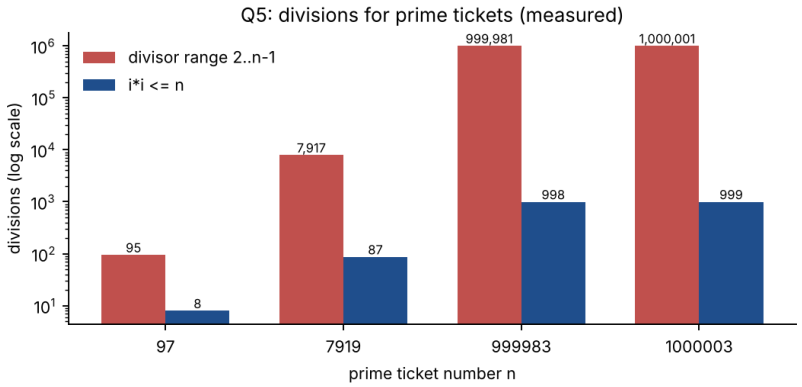


Figure 2: divisions for prime tickets, log scale. The gap is about 1000× at n = 10^6.

Observations. (1) The three loop constructs perform exactly the same number of divisions: the loop *form* does not change the work, the loop *condition* does. (2) For primes the improved count is $\lfloor\sqrt{n}\rfloor - 1$, matching the prediction table. (3) For composites the cost is set by the smallest factor (101 for 1000001), not by n. (4) Over every ticket 1 … 20,000 the naive loop performs 21,267,570 divisions and the improved loop 301,276 (about 70× fewer); over 1 … 1,000,003 the improved loop performs 67,741,504 divisions and all three constructs return identical answers for every n (0 mismatches).

4. Inquiry

4.1 Why the do-while version needs care for n = 2 and n = 3

A do-while executes its body **once before** the condition is examined. With i starting at 2 the body performs `n % 2` even though the principle says no division is needed when $i^2 = 4 > n$.

n	Unguarded do-while result	Divisions	Correct answer
2	**composite** (2 % 2 == 0)	1	prime — wrong
3	prime	1	prime (correct, but one needless division)
4	composite	1	composite
5	prime	1	prime (needless division)

Two correct remedies were tested: handle n < 4 before the loop (prime_dowhile in the appendix), or test the boundary at the top of the body `if (i > n / i) break;` as in the student program in Figure 4. Both give 0 divisions for 2 and 3 and identical answers for every n.

4.2 Why i * i <= n can overflow

If n and i are 32-bit unsigned values and n is close to 2^{32}, i^2 needs 33 bits once i = 65536. The product wraps modulo 2^{32} and the test becomes true again. Measured with n = 4,294,967,291 (the largest prime below 2^{32}):

```
i=65534 i*i (32-bit) = 4294705156 (i*i<=n is TRUE)
i=65535  i*i (32-bit) = 4294836225  (i*i<=n is TRUE)
i=65536 i*i  (32-bit)  =  0        (i*i<=n is TRUE)    <-- wrapped, loop should have stopped
i=65537 i*i (32-bit) = 131073      (i*i<=n is TRUE)
true 65536*65536 = 4294967296 (needs 33 bits)
Safe form i <= n/i: at i = 65536, n/i = 65535 -> condition FALSE (stops correctly)
```

After the wrap the loop no longer stops at $\sqrt{n}$; it can only end when some i divides n. For a prime this happens at i = n, so the loop would run about 4.3 billion divisions and report “not prime” (reasoned from the measured wrap; the full run was not executed). With signed int the overflow is undefined behaviour. In the assignment program `unsigned long` is 64-bit ($i^2 \le 1{,}000{,}003$ is safe), but the form `i <= n / i` is overflow-free for every n and costs the same number of divisions.

5. Alternative concept: Sieve of Eratosthenes

How the solution changes. Instead of testing each ticket separately, all numbers up to N are classified once. An array of N+1 flags is initialised to 1; for each prime i with $i^2 \le N$ all multiples from i^2 upward are set to 0. Afterwards each ticket is verified by one array lookup.

```
sieve(N):
    flag[0..N] = 1 ;  flag[0] = flag[1] = 0
    for i = 2 while i*i <= N:
        if flag[i] == 1:
            for j = i*i ; j <= N ; j += i:   flag[j] = 0

is_prime_query(n):  return flag[n]              /* O(1) */
```

Measured on N = 1,000,003 (file q5_sieve.c, same machine):

```
Sieve N=1000003: inner-loop writes=2122051, outer iterations=999, primes found=78499, time=0.0031 s,
    memory=1000004 bytes
Trial division for all n<=N: total divisions=67741504, mismatches vs sieve=0, time=0.2475 s
Average divisions per query (n uniform in 0..N) = 67.74
Break-even number of queries ~ 31326 (sieve writes / avg divisions)
1000 random tickets: trial divisions=64779, sieve lookups=1000
```

	Trial division (per ticket)	Sieve of Eratosthenes
Time, one query	$O(\sqrt{n})$ – 999 divisions for n = 1000003; O(1) for even n	O(1) lookup (after the sieve is built)
Time, preparation	none	O(N log log N): 2,122,051 writes measured for N = 1,000,003
Time, all N numbers	$\approx O(N\sqrt{N})$ worst case; measured 67,741,504 divisions	O(N log log N)
Space	O(1)	O(N): 1,000,004 bytes (one byte per flag; N/8 bytes if bit-packed)
Measured, whole range 0..N	0.245 s	0.0024 s (build) – results agree, 0 mismatches

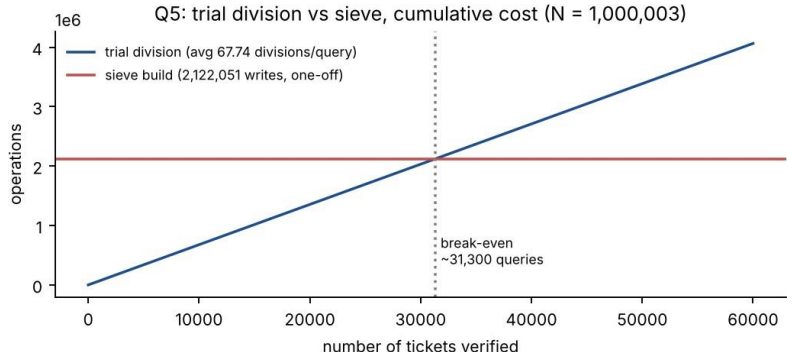


Figure 3: cumulative cost. The sieve pays off after about 31,300 randomly chosen queries (computed from the measured averages).

Trade-offs. For 1,000 random tickets trial division needs 64,779 divisions in total, which is *less* than the 2.1 million writes needed to build the sieve; for a few thousand verifications or an unknown, unbounded n, trial division is cheaper and needs no memory. When tickets are verified in bulk (tens of thousands of queries, or every number in a range) the sieve is far cheaper per query. The sieve also needs the upper limit N to be known in advance and memory proportional to N.

6. Student programs and execution evidence

The while and do-while programs below were executed by the student; their screenshots are reproduced as submitted. I re-compiled the same sources with `gcc -Wall` and tested n = 0, 1, 2, 3, 4, 9, 17, 19, 25, 97, 7919: every answer was correct, including n = 2 and 3, because the boundary `i <= n / i` is tested before the first division.

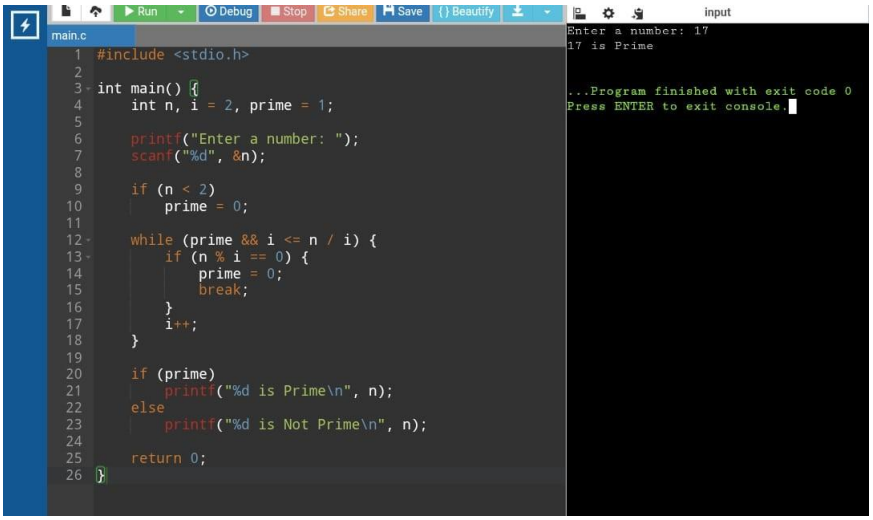


Figure 4a: while-loop version (condition: prime && i <= n / i), input 17

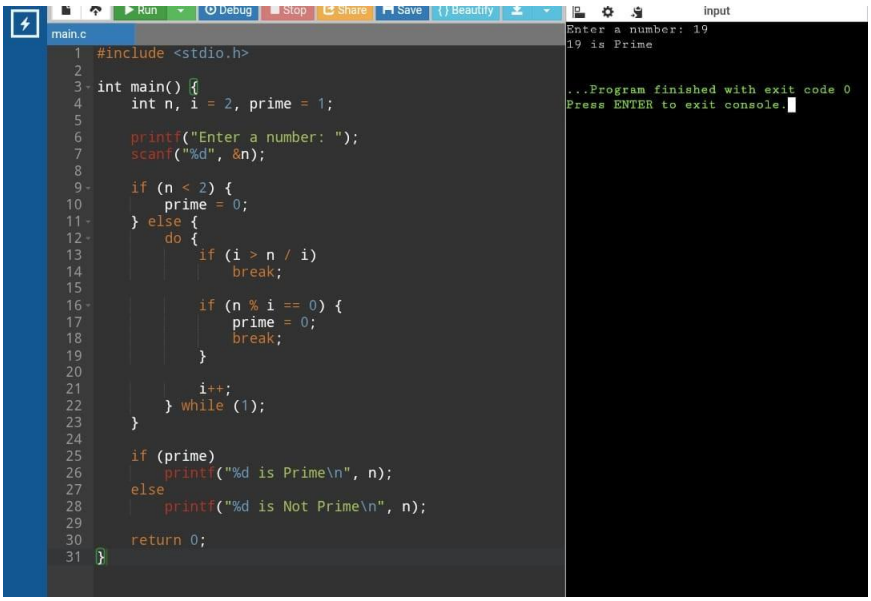


Figure 4b: do-while version (boundary test at the top of the body), input 19

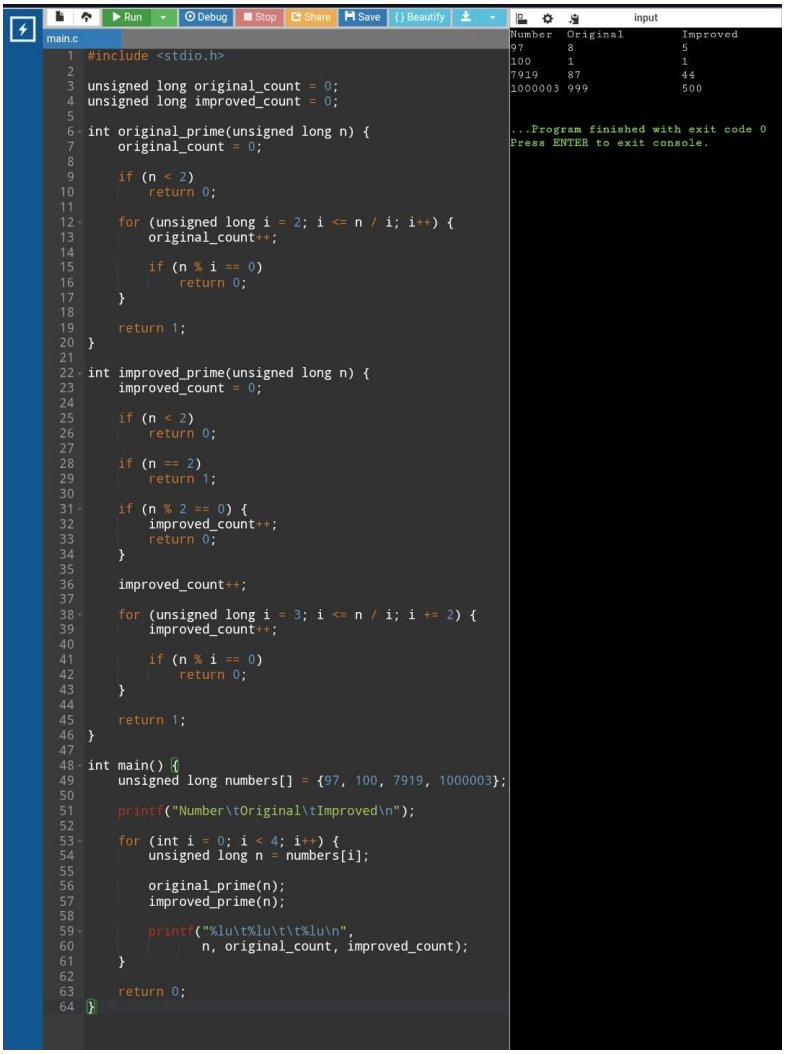


Figure 5: comparison program. 'Original' = improved loop (8, 1, 87, 999); 'Improved' = odd divisors only (5, 1, 44, 500).

7. Test table (normal, boundary and special cases)

Input n	Reason for the test	Expected	Observed	Divisions (i*i<=n)
0, 1	below 2	not prime	not prime	0
2, 3	smallest primes; do-while hazard	prime	prime	0
4	smallest composite	not prime	not prime	1
9, 25, 49	squares of primes (i = √n boundary)	not prime	not prime	2 / 4 / 6
97	prime, spec value	prime	prime	8
100	even	not prime	not prime	1
7919	prime, spec value	prime	prime	87
999983	largest prime below 10^6	prime	prime	998
1000001	composite with large smallest factor	not prime	not prime	100
1000003	large prime, spec value	prime	prime	999

The squares 9, 25 and 49 confirm that the condition must be i*i <= n (with equality): with `i * i < n` the divisor i = √n would be missed and these composites would be reported as prime.

8. Conclusion

Deriving the loop from the √n principle reduces the division count for the large prime 1000003 from 1,000,001 to 999 without changing any answer. The loop construct is a matter of style; the boundary condition, the early return, the do-while guard and the overflow-safe form of the test are what decide correctness and cost. For one-off checks the O(√n) test is preferred; for bulk verification the O(N log log N) sieve with O(N) memory is preferred.

Appendix A: source code (all executed with gcc 13.3.0)

q5_versions.c – naive, for, while, do-while, do-while without guard, overflow demo, totals

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

unsigned long checks;

/* V1: original version, divisors 2 .. n-1 (for) */
int prime_naive(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i < n; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* V2: improved, for loop, condition i <= n / i (no sqrt, no overflow) */
int prime_for(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* V3: improved, while loop */
int prime_while(unsigned long n){
    checks = 0; if (n < 2) return 0;
    unsigned long i = 2;
    while (i <= n / i) { checks++; if (n % i == 0) return 0; i++; }
    return 1;
}
/* V4: improved, do-while loop (needs the n < 4 guard) */
int prime_dowhile(unsigned long n){
    checks = 0; if (n < 2) return 0;
    if (n < 4) return 1;                /* n = 2, 3 handled before the loop */
    unsigned long i = 2;
    do { checks++; if (n % i == 0) return 0; i++; } while (i <= n / i);
    return 1;
}
/* V4-bug: do-while WITHOUT the guard */
int prime_dowhile_unguarded(unsigned long n){
    checks = 0; if (n < 2) return 0;
    unsigned long i = 2;
    do { checks++; if (n % i == 0) return 0; i++; } while (i * i <= n);
    return 1;
}

int main(void){
    unsigned long t[] = {2, 3, 97, 100, 7919, 999983, 1000001, 1000003};
    int nt = sizeof t / sizeof t[0];
    printf("== Division counts (checks) ==\n");
    printf("%-9s %-9s %10s %10s %10s %10s\n","n","result","naive","for","while","do-while");
    for (int k = 0; k < nt; k++){
        unsigned long n = t[k]; unsigned long c[4]; int r[4];
        r[0]=prime_naive(n); c[0]=checks;
        r[1]=prime_for(n); c[1]=checks;
        r[2]=prime_while(n); c[2]=checks;
        r[3]=prime_dowhile(n); c[3]=checks;
        int same = (r[0]==r[1] && r[1]==r[2] && r[2]==r[3]);
        printf("%-9lu %-9s %10lu %10lu %10lu %10lu %s\n", n, r[0]?"prime":"composite",
               c[0],c[1],c[2],c[3], same?"":"MISMATCH");
    }
    printf("\n== do-while without guard ==\n");
    for (unsigned long n = 2; n <= 5; n++){
        int r = prime_dowhile_unguarded(n);
        printf("n=%lu -> %s (divisions=%lu) [correct: %s]\n", n, r?"prime":"composite", checks,
               (n==2||n==3||n==5)?"prime":"composite");
    }
    printf("\n== 32-bit overflow of i*i <= n ==\n");
    {
        uint32_t n = 4294967291u; /* largest prime below 2^32 */
        uint32_t i; unsigned long iters = 0;
        for (i = 65534; iters < 6; i++, iters++){
            uint32_t sq = i * i;
            printf("i=%u  i*i (32-bit) = %u  (i*i<=n is %s)\n", i, sq, (sq <= n)?"TRUE":"FALSE");
        }
        printf("true i*i for 65536 = %llu (needs 33 bits)\n", 65536ULL*65536ULL);
        printf("Safe form i <= n/i: at i=65536, n/i = %u -> condition %s\n", n/65536u,
               (65536u <= n/65536u)?"TRUE":"FALSE");
    }
    printf("\n== Totals for every n = 1..1000003 ==\n");
    {
```

```
        unsigned long N = 1000003, tot_for = 0, tot_while=0, tot_do=0, bad=0;
        for (unsigned long n = 1; n <= N; n++){
            int a = prime_for(n); tot_for += checks;
            int b = prime_while(n); tot_while += checks;
            int c = prime_dowhile(n); tot_do += checks;
            if (a!=b || b!=c) bad++;
        }
        printf("total divisions: for=%lu while=%lu do-while=%lu, mismatches=%lu\n",
            tot_for,tot_while,tot_do,bad);
        unsigned long tot_naive = 0;
        for (unsigned long n = 1; n <= 20000; n++){ prime_naive(n); tot_naive += checks; }
        unsigned long tot_for20k = 0;
        for (unsigned long n = 1; n <= 20000; n++){ prime_for(n); tot_for20k += checks; }
        printf("n = 1..20000: naive total=%lu, sqrt total=%lu\n", tot_naive, tot_for20k);
    }
    return 0;
}
```

Output of q5_versions

```
== Division counts (checks) ==
n          result          naive        for       while    do-while
2          prime               0          0           0           0
3          prime               1          0           0           0
97         prime              95          8           8           8
100        composite           1          1           1           1
7919       prime            7917         87          87          87
999983     prime          999981        998         998         998
1000001    composite         100        100         100         100
1000003    prime          1000001        999         999         999

== do-while without guard ==
n=2 -> composite (divisions=1) [correct: prime]
n=3 -> prime (divisions=1) [correct: prime]
n=4 -> composite (divisions=1) [correct: composite]
n=5 -> prime (divisions=1) [correct: prime]

== 32-bit overflow of i*i <= n ==
i=65534  i*i (32-bit) = 4294705156  (i*i<=n is TRUE)
i=65535  i*i (32-bit) = 4294836225  (i*i<=n is TRUE)
i=65536  i*i (32-bit) = 0  (i*i<=n is TRUE)
i=65537  i*i (32-bit) = 131073  (i*i<=n is TRUE)
i=65538  i*i (32-bit) = 262148  (i*i<=n is TRUE)
i=65539  i*i (32-bit) = 393225  (i*i<=n is TRUE)
true i*i for 65536 = 4294967296 (needs 33 bits)
Safe form i <= n/i: at i=65536, n/i = 65535 -> condition FALSE

== Totals for every n = 1..1000003 ==
total divisions: for=67741504 while=67741504 do-while=67741504, mismatches=0
n = 1..20000: naive total=21267570, sqrt total=301276
```

q5_sieve.c – sieve of Eratosthenes versus trial division

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
unsigned long checks;
int prime_for(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec + t.tv_nsec*1e-9;
     }
int main(void){
    const unsigned long N = 1000003;
    unsigned char *flag = malloc(N + 1);
    unsigned long marks = 0, scan = 0;
    double t0 = now();
    for (unsigned long i = 0; i <= N; i++) flag[i] = 1;
    flag[0] = flag[1] = 0;
    for (unsigned long i = 2; i * i <= N; i++){
        scan++;
        if (flag[i]) for (unsigned long j = i * i; j <= N; j += i){ flag[j] = 0; marks++; }
    }
    double t_sieve = now() - t0;
    unsigned long primes = 0; for (unsigned long i = 0; i <= N; i++) primes += flag[i];
    printf("Sieve N=%lu: inner-loop writes=%lu, outer iterations=%lu, primes found=%lu, time=%.4f s, memory=%lu
        bytes\n",
           N, marks, scan, primes, t_sieve, N + 1);
    /* correctness + cost of trial division for every n <= N */
    unsigned long tot = 0, bad = 0;
    t0 = now();
    for (unsigned long n = 0; n <= N; n++){ int r = prime_for(n); tot += checks; if (r != flag[n]) bad++; }
    double t_trial = now() - t0;
    printf("Trial division for all n<=N: total divisions=%lu, mismatches vs sieve=%lu, time=%.4f s\n", tot, bad,
         t_trial);
    printf("Average divisions per query (n uniform in 0..N) = %.2f\n", (double)tot/(N+1));
    printf("Break-even number of queries ~ %.0f (sieve writes / avg divisions)\n",
        (double)marks/((double)tot/(N+1)));
    /*  1000 sample tickets */
    srand(12345); unsigned long q1000 = 0;
    for (int k = 0; k < 1000; k++){ unsigned long n = 2 + (unsigned long)rand() % (N - 1); prime_for(n); q1000
        += checks; }
    printf("1000 random tickets: trial divisions=%lu, sieve lookups=1000\n", q1000);
    free(flag); return 0;
}
```

Question 6: Loops That Must Terminate: Loan Repayment Schedule

Student name	_____
Register number	_____
GitHub repository	CSA02-<RegisterNumber>-Assignment2 (folder: Q6)
Link	_____
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compile command	`gcc -Wall -Wextra -O2 -o q6_loan q6_loan.c -lm`
Marks / Bloom level	8 marks / K5 – Evaluate

1. Problem and loop design

A self-help group lends money at a monthly interest rate. Each month the interest is added to the outstanding balance and the member repays a fixed amount E. The program prints the month-by-month schedule until the loan is fully repaid and reports the number of months and the total interest paid. Notation: P = loan, r = monthly rate (r = 2% means 0.02), E = fixed monthly repayment.

1.1 Which loop?

Loop	Fit for this problem?	Reason
for	No	The number of months is the unknown result, not a known count. A for loop would hide the real stopping rule inside a fake bound.
while	**Yes (chosen)**	The rule is a condition ("money is still owed"). It is tested *before* each month, so an already-zero balance runs zero months and no negative payment is produced.
do-while	No	It always runs the body once, even when there is nothing to repay.

1.2 Termination condition and the floating-point trap

The loop continues while `balance > 0.005` (half a paisa). The test `balance != 0` is dangerous because a double cannot represent most decimal amounts exactly, so a balance that is mathematically zero is often a tiny non-zero value (or slightly negative). Experiments (file q6_experiments.c):

Experiment	Result	Meaning
P = 1.00, r = 0, E = 0.10, payment clamped, test **balance != 0**	11 months (should be 10)	After 10 payments balance = 1.4×10^{-16}, not 0; an extra month is needed
same, test **balance > 0.005**	10 months	correct
P = 1.00, E = 0.10, no clamp, test **balance != 0**	never stops (cap at 50 months, balance − 4.0)	balance jumps over 0 to a negative value
P = 10000, 2%, E = 2000, no clamp, test **balance != 0**	cap at 360 months, balance − 112,180,501.50	the loan is "overpaid" forever; the loop never ends

1.3 The last month and an inadequate repayment

- **Last month.** When the balance plus interest is smaller than E, the member pays only what is left: `payment = (repayment < balance) ? repayment : balance;`. Without this clamp the final balance would be negative (an overpayment) and the total paid would be wrong.
- **E smaller than the monthly interest.** The balance then *grows* every month and the loop would never end. If E equals the interest, the balance stays constant (interest-only) and it also never ends. The program therefore stops with a message when `repayment <= interest;` in the example below E = 200 = interest and the program stops after month 1.

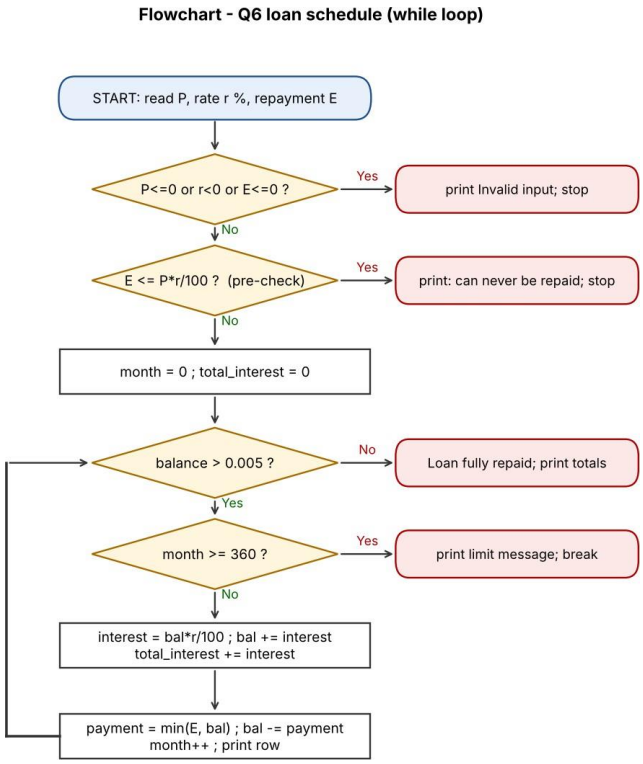


Figure 1: flowchart of the final design (while loop, pre-check, tolerance, 360-month limit)

2. Student program and execution

The program written by the student (screenshot below) uses the while loop, the 0.005 tolerance, the clamped last payment, the `repayment <= interest` test and the 360-month limit. Source in Appendix A.

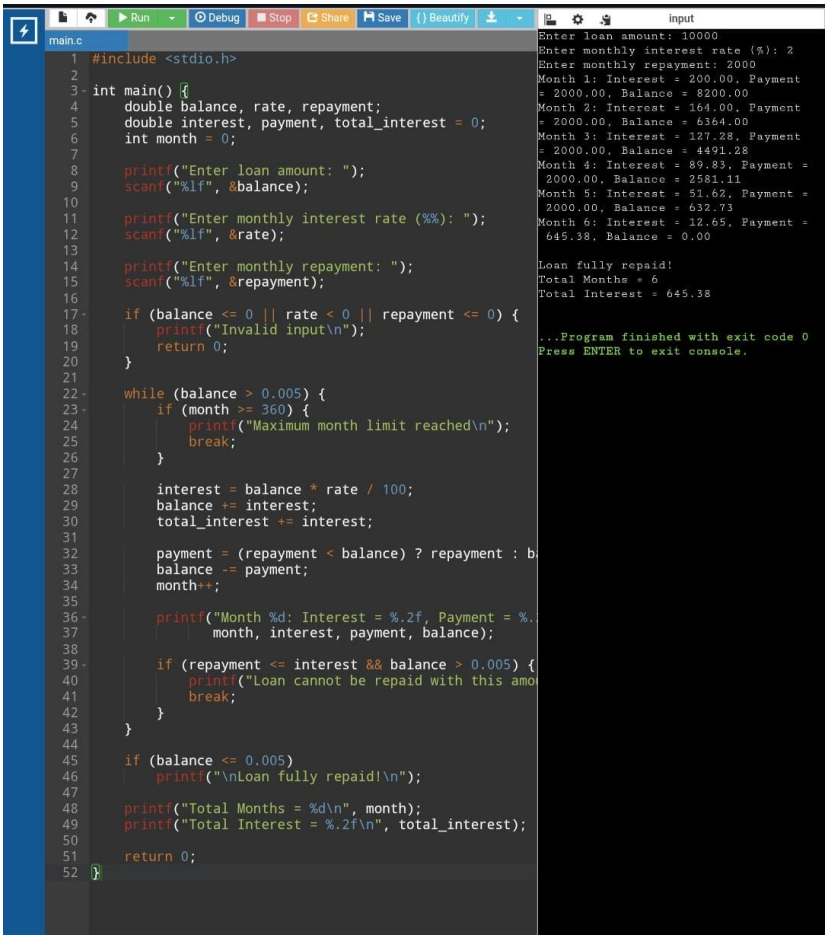


Figure 2: student program and its output for Rs 10,000, 2% per month, Rs 2,000 repayment

3. Correctness check: hand schedule versus program

Hand calculation for P = Rs 10,000, r = 2% per month, E = Rs 2,000 (interest = balance × 0.02; new balance = balance + interest − payment):

Month	Opening balance	Interest (hand)	Balance + interest	Payment	Closing balance (hand)	Program output
1	10,000.0000	200.0000	10,200.0000	2,000.00	8,200.0000	8200.00
2	8,200.0000	164.0000	8,364.0000	2,000.00	6,364.0000	6364.00
3	6,364.0000	127.2800	6,491.2800	2,000.00	4,491.2800	4491.28
4	4,491.2800	89.8256	4,581.1056	2,000.00	2,581.1056	2581.11
5	2,581.1056	51.6221	2,632.7277	2,000.00	632.7277	632.73
6	632.7277	12.6546	645.3823	**645.38** (last)	0.0000	0.00

Total interest by hand = 200 + 164 + 127.28 + 89.8256 + 51.6221 + 12.6546 = **645.3823**; months = **6**. Cross-check: total paid = 5 × 2,000 + 645.38 = 10,645.38 = loan + interest. The program printed **Total Months = 6** and **Total Interest = 645.38**: the hand schedule and the program agree in every row.

```
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Month 1: Interest = 200.00, Payment
     = 2000.00, Balance = 8200.00
Month 2: Interest = 164.00, Payment = 2000.00, Balance = 6364.00
Month 3: Interest = 127.28, Payment = 2000.00, Balance = 4491.28
Month 4: Interest = 89.83, Payment = 2000.00, Balance = 2581.11
Month 5: Interest = 51.62, Payment = 2000.00, Balance = 632.73
Month 6: Interest = 12.65, Payment = 645.38, Balance = 0.00

Loan fully repaid!
Total Months = 6
Total Interest = 645.38
```

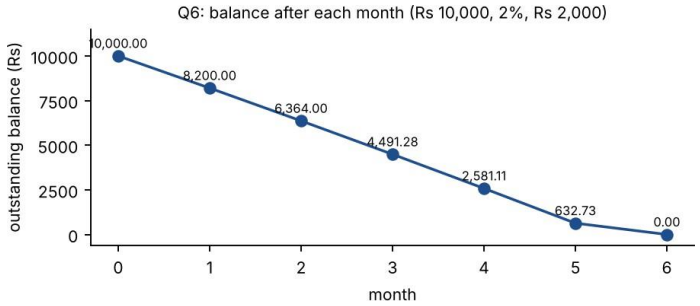


Figure 3: outstanding balance after each month

4. Growth investigation: repayment approaching the interest

Prediction. Solving the recurrence gives $n = -\ln(1 - rP/E) / \ln(1 + r)$. With $rP = 200$ the argument $1 - 200/E$ tends to 0 as E falls towards 200, so n grows without bound (logarithmically in $1/(E - 200)$). Predicted: moderate growth for large E, rapid growth near $E = 200$; every time $E - 200$ is divided by 5, about $\ln 5 / \ln 1.02 \approx 81$ extra months. Measured (simulation, file q6_experiments.c):

E (Rs)	Months (loop iterations)	Predicted n (formula)	ceil(n)	Total interest (Rs)
5000	3	2.061	3	310.08
3000	4	3.484	4	459.50
2000	6	5.321	6	645.38
1500	8	7.226	8	842.14
1000	12	11.268	12	1,270.33
700	17	16.991	17	1,893.96
500	26	25.796	26	2,898.73
400	36	35.003	36	4,001.13
300	56	55.478	56	6,644.17
250	82	81.274	82	10,318.98
220	122	121.090	122	16,639.92
205	188	187.529	188	28,444.00
201	268	267.808	268	43,829.75
200.5	303	302.685	303	50,688.80
200.1	> 360 (limit hit, Rs 3,767.19 left)	383.858	384	65,803.19 at month 360
200	never (balance stays 10,000)	undefined	–	–
150	never (balance grows)	undefined	–	–

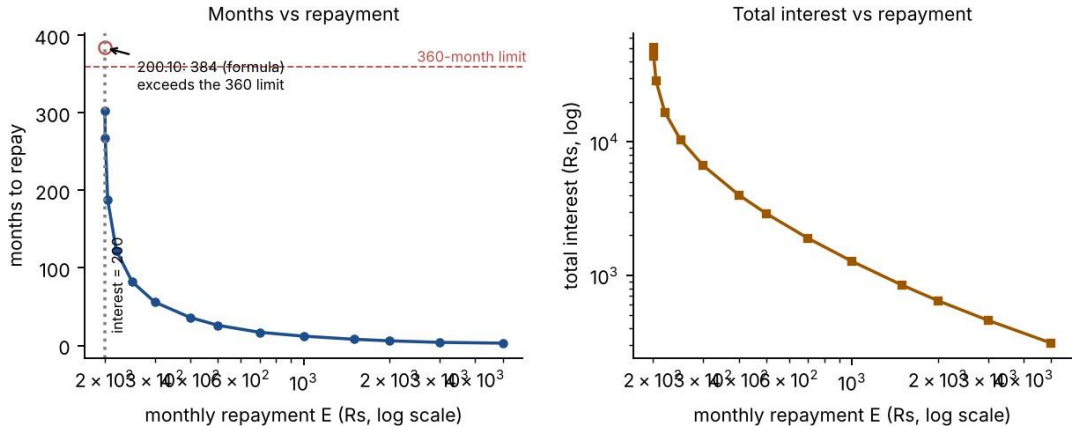


Figure 4: months and total interest against the repayment (measured)

Observations. Halving E from 2,000 to 1,000 doubles the months (6 → 12) and almost doubles the interest paid. Between 400 and 250 the months more than double again; between 205 and 200.5 the schedule stretches from 188 to 303 months, and at 200.1 it passes the 360-month limit. The measured steps agree with the prediction (E−200 = 5 → 1: +80 months, predicted 81). The ceil of the formula equals the simulated number of months for every row.

5. Deployment decision: safety limit and messages

The limit of **360 months** (30 years) is implemented with break. Recommended final behaviour (file q6_loan_final.c, same schedule output as above):

Situation	Message / action	Reason
Invalid input (P ≤ 0, r < 0, E ≤ 0)	"Invalid input", exit	protects the arithmetic
E ≤ first month's interest	"Repayment Rs X does not exceed the first month's interest Rs Y: loan can never be repaid." (checked **before** the loop)	detects the impossible loan at once instead of printing a misleading month 1 or waiting 360 iterations
Still unpaid after 360 months	"Not repaid within 360 months. Outstanding balance Rs B. Increase the repayment."; the totals are still printed; "Loan fully repaid" is **not** printed	safety net for borderline loans (E = 200.1 needs 384 months) and for any rounding or logic error

```
### Input: P=10000 rate=2% E=2000
Month 2: Interest = 164.00, Payment = 2000.00, Balance = 6364.00
Month 3: Interest = 127.28, Payment = 2000.00, Balance = 4491.28
Month 4: Interest = 89.83, Payment = 2000.00, Balance = 2581.11
Month 5: Interest = 51.62, Payment = 2000.00, Balance = 632.73
Month 6: Interest = 12.65, Payment = 645.38, Balance = 0.00

Loan fully repaid!
Total Months = 6
Total Interest = 645.38

### Input: P=10000 rate=2% E=200
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Repayment Rs 200.00 does not exceed
     the first month's interest Rs 200.00: loan can never be repaid.

### Input: P=10000 rate=2% E=150
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Repayment Rs 150.00 does not exceed
     the first month's interest Rs 200.00: loan can never be repaid.

### Input: P=10000 rate=2% E=200.1
Month 360: Interest = 77.79, Payment = 200.10, Balance = 3767.19
Not repaid within 360 months. Outstanding balance Rs 3767.19. Increase the repayment.
Total Months = 360
Total Interest = 65803.19

### Input: P=5000 rate=0% E=1000
Month 2: Interest = 0.00, Payment = 1000.00, Balance = 3000.00
Month 3: Interest = 0.00, Payment = 1000.00, Balance = 2000.00
Month 4: Interest = 0.00, Payment = 1000.00, Balance = 1000.00
Month 5: Interest = 0.00, Payment = 1000.00, Balance = 0.00

Loan fully repaid!
Total Months = 5
Total Interest = 0.00

### Input: P=500 rate=2% E=1000
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Month 1: Interest = 10.00, Payment
    = 510.00, Balance = 0.00

Loan fully repaid!
Total Months = 1
Total Interest = 10.00

### Input: P=0 rate=2% E=1000
Enter loan amount: Enter monthly interest rate (%): Enter monthly repayment: Invalid input
```

Defence. (1) The pre-check gives the member a precise reason and the exact minimum needed (more than Rs 200); the 360-month limit alone would only say "something is wrong" after printing hundreds of lines. (2) The limit is still kept because it is the only guard against loops whose end depends on floating-point behaviour and against loans that are technically repayable but beyond any

realistic horizon. (3) 360 months is long compared with micro-loans (typically a few months to a few years) and is measured: the E = 200.1 case shows it triggers with Rs 3,767.19 outstanding. (4) The message states the outstanding balance and the remedy, and the program never claims the loan is repaid when it is not.

6. Alternative concept: direct formula

How the solution changes. The number of months is computed directly: $n = -\log(1 - rP/E) / \log(1 + r)$, and the months are ceil(n). No loop and no schedule are produced. The total interest needs the last payment: after $k = \text{months} - 1$ full payments the balance is $B_k = P(1+r)^k - E((1+r)^k - 1)/r$, the last payment is $B_k(1+r)$, and total interest $= k \cdot E + B_k(1+r) - P$.

```
double x = r * P / E;                    /* r as a fraction, e.g. 0.02 */
if (x >= 1.0) { puts("never repaid"); }
else {
    double n = -log1p(-x) / log1p(r);     /* stable form of -log(1-x)/log(1+r) */
    int months = (int)ceil(n - 1e-9);     /* guard against 6.0000000001 */
}
```

Aspect	Simulation loop	Closed formula
Time	O(n) iterations, n = months (3 ... 303 measured; 360 max)	O(1): two logarithms
Space	O(1)	O(1)
Output	full schedule, exact last payment, total interest by summation	months only (schedule needs the loop anyway)
Precision	each month adds rounding error (about $n \cdot \epsilon$ in total); needs a tolerance such as 0.005	ceil() is fragile at exact integers: the exact 6-month payment E = 1785.2581233520 gives n = 5.999999999999999 (ceil = 6, correct) but a value like 6.0000000000000009 would give 7; needs a small epsilon
Edge cases	r = 0 works (5,000 ÷ 1,000 = 5 months); $E \le rP$ detected by comparison	r = 0 gives log(1) = 0 and 0/0 = NaN (measured); $E \le rP$ makes the logarithm undefined and must be tested first

Measured agreement: for all 14 loans that terminate within 360 months, ceil(formula) equals the simulated months (table in section 4). When $1 - rP/E$ is very small, the plain form `log(1 - x)` loses digits ($E = 200.0001$: $1 - x = 5.0 \times 10^{-7}$), so `log1p(-x)` is preferred; for these values both forms still printed 732.663299, so the loss stayed below 10^{-6} in double precision.

Recommendation. Use the loop to produce the schedule (the assignment requires it), and use the formula as a O(1) feasibility check and an independent cross-check of the number of months. Neither replaces the other.

7. Test table

Input (P, r %, E)	Purpose	Expected	Observed
10000, 2, 2000	spec example, hand schedule	6 months, interest 645.38	6 months, 645.38
10000, 2, 200	E = interest	refused, cannot be repaid	message printed, no endless loop
10000, 2, 150	E < interest	refused	message printed
10000, 2, 200.1	just above interest, long loan	limit message at 360	limit message, Rs 3,767.19 left
5000, 0, 1000	zero interest	5 months, interest 0	5 months, 0.00
500, 2, 1000	E larger than balance + interest	1 month, payment 510.00	1 month, payment 510.00
0, 2, 1000	invalid	Invalid input	Invalid input

8. Conclusion

A while loop with a tolerance-based condition, a clamped last payment, a check for repayments that cannot cover the interest and a 360-month limit terminates for every input tested. The hand schedule matches the program in every month; the iteration counts grow slowly and then explosively as E approaches the interest, which is the reason both the pre-check and the limit are justified.

Appendix A: source code (executed with gcc 13.3.0)

q6_loan.c – student program (while loop, tolerance, limit)

```
#include <stdio.h>
#include <math.h>

/* Loan schedule: while loop, tolerance test, clamped last payment, 360-month safety limit */
int main(void) {
    double balance, rate, repayment;
    double interest, payment, total_interest = 0;
    int month = 0;

    printf("Enter loan amount: ");             scanf("%lf", &balance);
    printf("Enter monthly interest rate (%%): "); scanf("%lf", &rate);
    printf("Enter monthly repayment: ");  scanf("%lf", &repayment);

    if (balance <= 0 || rate < 0 || repayment <= 0) { printf("Invalid input\n"); return 0; }

    while (balance > 0.005) {
        if (month >= 360) { printf("Maximum month limit reached\n"); break; }
        interest = balance * rate / 100;
        balance   +=   interest;
        total_interest += interest;
        payment = (repayment < balance) ? repayment : balance;
        balance -= payment;
        month++;
        printf("Month %d: Interest = %.2f, Payment = %.2f, Balance = %.2f\n",
               month, interest, payment, balance);
        if (repayment <= interest && balance > 0.005) {
            printf("Loan cannot be repaid with this amount\n");
            break;
        }
    }
    if (balance <= 0.005) printf("\nLoan fully repaid!\n");
    printf("Total Months = %d\n", month);
    printf("Total Interest = %.2f\n", total_interest);
    return 0;
}
```

q6_loan_final.c – recommended deployment version

```c
#include <stdio.h>
#define MAX_MONTHS 360    /* safety limit: 30 years */

int main(void) {
    double balance, rate, repayment, interest, payment, total_interest = 0.0;
    int month = 0;

    printf("Enter loan amount: ");
    if (scanf("%lf", &balance) != 1) return 1;
    printf("Enter monthly interest rate (%%): ");
    if (scanf("%lf", &rate) != 1) return 1;
    printf("Enter monthly repayment: ");
    if (scanf("%lf", &repayment) != 1) return 1;

    if (balance <= 0 || rate < 0 || repayment <= 0) { printf("Invalid input\n"); return 0; }

    /* Pre-check: if the repayment cannot even cover the first month's interest, the balance never falls */
    interest = balance * rate / 100.0;
    if (repayment <= interest) {
        printf("Repayment Rs %.2f does not exceed the first month's interest Rs %.2f: loan can never be
            repaid.\n",
               repayment, interest);
        return 0;
    }

    while (balance > 0.005) {                  /* tolerance: half a paisa */
        if(month >= MAX_MONTHS) {
            printf("Not repaid within %d months. Outstanding balance Rs %.2f. Increase the repayment.\n",
                   MAX_MONTHS, balance);
            break;
        }
        interest = balance * rate / 100.0;
        balance   +=   interest;
        total_interest += interest;
        payment = (repayment < balance) ? repayment : balance; /* last month: pay only what is left */
        balance -= payment;
        month++;
        printf("Month %d: Interest = %.2f, Payment = %.2f, Balance = %.2f\n", month, interest, payment,
            balance);
    }
    if (balance <= 0.005) printf("\nLoan fully repaid!\n");
    printf("Total Months = %d\n", month);
    printf("Total Interest = %.2f\n", total_interest);
    return 0;
}
```

q6_experiments.c – growth, floating-point and formula experiments

```c
#include <stdio.h>
#include <math.h>

/* returns months, or -1 if limit reached; iterations = months */
static int simulate(double P, double r_pct, double E, double *tot_int, double *final_bal){
    double bal = P, ti = 0; int m = 0;
    while (bal > 0.005) {
        if (m >= 360) { *tot_int = ti; *final_bal = bal; return -1; }
        double in = bal * r_pct / 100; bal += in; ti += in;
        double pay = (E < bal) ? E : bal; bal -= pay; m++;
    }
    *tot_int = ti; *final_bal = bal; return m;
}
int main(void){
    double P = 10000, r = 2.0;
    printf("== Growth investigation: P=10000, r=2%% (interest in month 1 = 200) ==\n");
    printf("%10s %10s %12s %14s %14s\n","repayment","months","formula n","ceil(formula)","total interest");
    double E[] = {5000,3000,2000,1500,1000,700,500,400,300,250,220,205,201,200.5,200.1,200,150};
    int nE = sizeof E / sizeof E[0];
    for (int k = 0; k < nE; k++){
        double ti, fb; int m = simulate(P, r, E[k], &ti, &fb);
        double rr = r/100.0, arg = 1.0 - rr*P/E[k];
        char fs[32] = "n/a", cs[32] = "n/a";
        if (arg > 0) { double n = -log(arg)/log(1+rr); snprintf(fs,32,"%.3f",n); snprintf(cs,32,"%.0f",ceil(n));
            }
        if (m < 0) printf("%10.2f %10s %12s %14s(limit; balance left %.2f)\n", E[k], ">360", fs, cs, fb);
        else printf("%10.2f %10d %12s %14s %14.2f\n", E[k], m, fs, cs, ti);
    }
    /* exact hand-check numbers */
    printf("\n== Floating-point danger: balance != 0 ==\n");
    {   /* rate 0, P=1.0, E=0.1 : should take exactly 10 months */
        double bal = 1.0; int m = 0;
        while (bal != 0 && m < 50) { double pay = (0.1 < bal) ? 0.1 : bal; bal -= pay; m++; }
        printf("Clamped payment, test 'balance != 0': months=%d (expected 10), balance=%.3e\n", m, bal);
        bal = 1.0; m = 0;
        while (bal > 0.005 && m < 50) { double pay = (0.1 < bal) ? 0.1 : bal; bal -= pay; m++; }
        printf("Clamped payment, test 'balance > 0.005': months=%d\n", m);
        bal = 1.0; m = 0;
        while (bal != 0 && m < 50) { bal -= 0.1; m++; }
        printf("No clamp, test 'balance != 0': stopped by cap at months=%d, balance=%.3e (never exactly 0)\n",
            m, bal);
        bal = 10000; m = 0;
        while (bal != 0 && m < 360) { bal += bal*0.02; bal -= 2000; m++; }
        printf("Real loan, no clamp, 'balance != 0': cap hit at m=%d, balance=%.2f (negative: overpaid, never
            0)\n", m, bal);
    }
    printf("\n== Formula vs simulation precision ==\n");
    {
        double rr = 0.02;
        double Evs[] = {200.0001, 200.01, 200.4};
        for (int q = 0; q < 3; q++) {
            double Ev = Evs[q];
            double x = rr*P/Ev;
            double n_naive = -log(1.0 - x)/log(1.0 + rr);
            double n_stable = -log1p(-x)/log1p(rr);
            printf("E=%.4f: naive formula n=%.6f, log1p form n=%.6f, 1-x=%.3e\n", Ev, n_naive, n_stable, 1.0-x);
        }
        /* integer-valued n : choose E so n is exactly 6? search E producing n close to integer */
        double Ev6 = rr*P / (1 - pow(1+rr, -6.0));  /* payment that clears loan in exactly 6 months */
        double n6 = -log(1.0 - rr*P/Ev6)/log(1.0+rr);
        double ti, fb; int m = simulate(P, 2.0, Ev6, &ti, &fb);
        printf("E=%.10f (exact 6-month annuity): formula n=%.15f, ceil=%.0f, simulation months=%d, final
            bal=%.3e\n", Ev6, n6, ceil(n6), m, fb);
        printf("E=%.2f (rounded to paise): formula n=%.6f, simulation months=%d\n", round(Ev6*100)/100, -log(1.0
             - rr*P/(round(Ev6*100)/100))/log(1.0+rr), simulate(P,2.0,round(Ev6*100)/100,&ti,&fb));
        printf("r=0: formula divides by log(1)=%.1f -> n = %f (inf/nan); simulation handles it.\n", log(1.0),
            -log(1.0-0.0)/log(1.0+0.0));
    }
    return 0;
}
```

Output of q6_experiments

```
== Growth investigation: P=10000, r=2% (interest in month 1 = 200) ==
  repayment      months    formula n    ceil(formula)total interest
     5000.00          3         2.061              3          310.08
     3000.00          4         3.484              4          459.50
     2000.00          6         5.321              6          645.38
     1500.00          8         7.226              8          842.14
     1000.00         12        11.268             12         1270.33
      700.00         17        16.991             17         1893.96
      500.00         26        25.796             26         2898.73
      400.00         36        35.003             36         4001.13
      300.00         56        55.478             56         6644.17
      250.00         82        81.274             82        10318.98
      220.00        122       121.090            122        16639.92
      205.00        188       187.529            188        28444.00
      201.00        268       267.808            268        43829.75
      200.50        303       302.685            303        50688.80
      200.10       >360       383.858            384   (limit; balance left 3767.19)
      200.00       >360           n/a            n/a   (limit; balance left 10000.00)
      150.00       >360           n/a            n/a   (limit; balance left 3126402.82)

== Floating-point danger: balance != 0 ==
Clamped payment, test 'balance != 0': months=11 (expected 10), balance=0.000e+00
Clamped payment, test 'balance > 0.005': months=10
No clamp, test 'balance != 0': stopped by cap at months=50, balance=-4.000e+00 (never exactly 0)
Real loan, no clamp, 'balance != 0': cap hit at m=360, balance=-112180501.50 (negative: overpaid, never 0)

== Formula vs simulation precision ==
E=200.0001: naive formula n=732.663299, log1p form n=732.663299, 1-x=5.000e-07
E=200.0100: naive formula n=500.112303, log1p form n=500.112303, 1-x=5.000e-05
E=200.4000: naive formula n=313.928349, log1p form n=313.928349, 1-x=1.996e-03
E=1785.2581233520 (exact 6-month annuity): formula n=5.999999999999999, ceil=6, simulation months=6, final
    bal=7.503e-12
E=1785.26 (rounded to paise): formula n=5.999993, simulation months=6
r=0: formula divides by log(1)=0.0 -> n = -nan (inf/nan); simulation handles it.
```

C PROGRAMMING – ASSIGNMENT 2 (CO2)

Question 7: Day-4 Design Challenge: Diagnose, Redesign and Validate

Student name	_____
Register number	_____
GitHub repository	CSA02-<RegisterNumber>-Assignment2 (folder: Q7)
Link	_____
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compile command	`gcc -Wall -Wextra -O2 -o q7_redesign q7_redesign.c`
Marks / Bloom level	13 marks / K5 – Evaluate

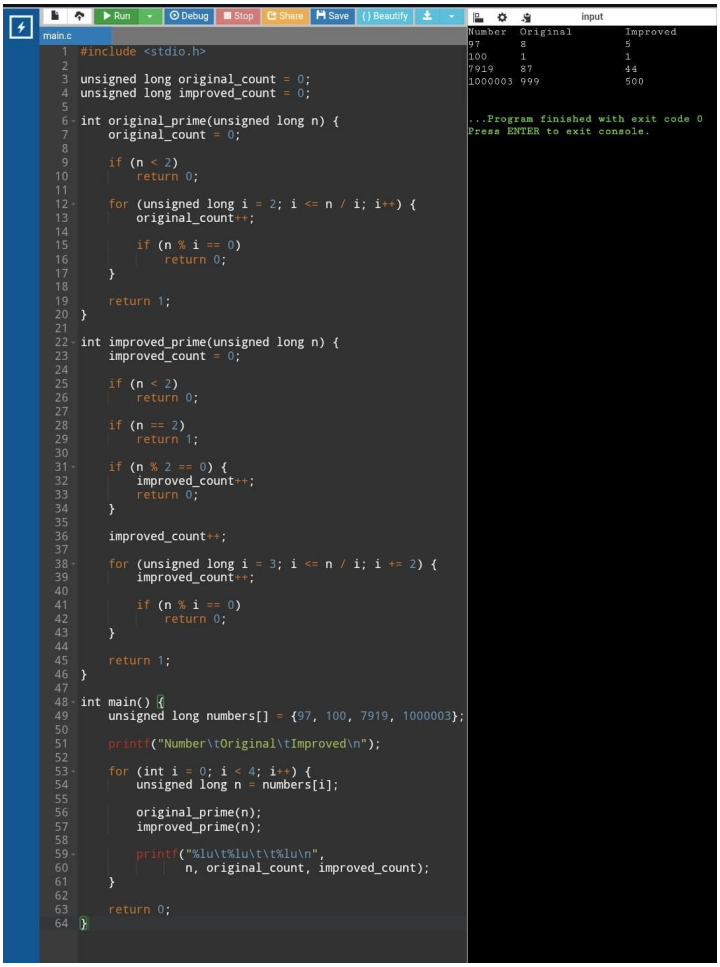
Selected program

Program chosen: Q5 – prime test for the lucky-draw tickets (trial division with the loop condition `i * i <= n`, called the *original* below). It is the program from Q1–Q6 whose cost visibly grows with the input: 999 divisions for the ticket 1000003, against 8 for 97. The change is **not** a switch between for, while and do-while – Q5 showed that the loop form leaves the division count unchanged. The redesign exploits a number-theoretic property of the divisors that actually need testing.

A. Diagnose the bottleneck (2 marks)

All figures come from the Q5 programs (appendix of this report, q7_redesign.c) over the tickets 2 … 1,000,003.

Question	Evidence from the original program
Where are most operations performed?	78,499 prime tickets (7.8% of the numbers) cause **50,659,938 of the 67,741,504 divisions (74.8%)**. A prime must survive every divisor up to $\sqrt{n}$; a composite stops at its smallest factor. Even tickets cost 1 division.
Which count grows with the input?	The number of divisions for a prime is $\lfloor\sqrt{n}\rfloor - 1$: **8 (n = 97), 87 (7919), 998 (999983), 999 (1000003)**. The Q5 screenshot (Figure 1) shows the same 8 / 1 / 87 / 999. It grows with $\sqrt{n}$: ten times more digits of work for every 100× larger ticket.
Is work unnecessarily repeated?	Yes. Of the 66,820,001 divisions that did not find a divisor, **33,640,395 (50.3%) used an even i** and **11,220,986 (16.8%) used an odd multiple of 3**: **67.1%** of all divisions test a divisor that cannot divide n once 2 and 3 have been tried (an even i dividing n would mean 2 divides n). Every ticket also re-tests 2, 3, 4, 5 … from scratch, sharing nothing with the previous ticket.




```c
#include <stdio.h>

unsigned long original_count = 0;
unsigned long improved_count = 0;

int original_prime(unsigned long n) {
    original_count = 0;

    if (n < 2)
        return 0;

    for (unsigned long i = 2; i <= n / i; i++) {
        original_count++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int improved_prime(unsigned long n) {
    improved_count = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    if (n % 2 == 0) {
        improved_count++;
        return 0;
    }

    improved_count++;

    for (unsigned long i = 3; i <= n / i; i += 2) {
        improved_count++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int main() {
    unsigned long numbers[] = {97, 100, 7919, 1000003};

    printf("Number\tOriginal\tImproved\n");

    for (int i = 0; i < 4; i++) {
        unsigned long n = numbers[i];

        original_prime(n);
        improved_prime(n);

        printf("%lu\t%lu\t\t%lu\n",
               n, original_count, improved_count);
    }

    return 0;
}
```

Figure 1: Q5 comparison program output. 'Original' column = the program being diagnosed (8, 1, 87, 999); 'Improved' column = odd divisors only (5, 1, 44, 500), the first step of the redesign.

B. Design objective (2 marks)

The objective is measurable and derived from the diagnosis (67% of divisions are redundant):

#	Objective	How it is measured
O1	Reduce divisions for the worst-case ticket 1000003 from 999 to **≤ 350** (at least 65% fewer)	division counter (same counter as Q5)
O2	Reduce the **total** divisions over all tickets 2 … 1,000,003 by **≥ 60%** (67.74 M → ≤ 27.1 M)	sum of the counter over the range
O3	**Identical** prime / not-prime answer for every ticket	compare with the original for all 1,000,002 values
O4	**No extra memory**: O(1) space	no arrays
O5	Never noticeably worse than the original	compare the counter value for every n

C. Design of the revised solution (4 marks)

C.1 What changes and which property is used

Every integer is 6k, 6k+1, 6k+2, 6k+3, 6k+4 or 6k+5. The forms 6k, 6k+2 and 6k+4 are even and 6k+3 is divisible by 3. Hence, **once n has been tested for 2 and 3, any prime factor of n must be of the form 6k−1 or 6k+1** (5, 7, 11, 13, 17, 19, 23, 25 …). The smallest factor of a composite n is itself prime, so it is always among these candidates. The redesign therefore (1) tests 2 and 3 once, (2) tests only i = 6k−1 and i+2 = 6k+1 in each pass and advances i by 6, (3) keeps the early return and the condition `i * i <= n`. Only 2 of every 6 integers are tried (one third). This is a change of *which divisors are tested*, not of the loop statement.

Stages. R1 = test 2, then odd divisors only (half of the candidates; already visible in the student's Q5 screenshot). R2 = 6k±1 (one third) = the final redesign.

C.2 Pseudocode

```
is_prime_6k(n):
    checks = 0
    if n < 2 : return 0
    if n < 4 : return 1                  # 2 and 3 are prime
    checks++ ; if n mod 2 == 0 : return 0
    checks++ ; if n mod 3 == 0 : return 0
    i = 5
    while i * i <= n:
        checks++ ; if n mod i       == 0 : return 0      # candidate 6k-1
        checks++ ; if n mod (i + 2) == 0 : return 0      # candidate 6k+1
        i = i + 6
    return 1
```

Flowchart - Q7 redesigned test (6k +/- 1)

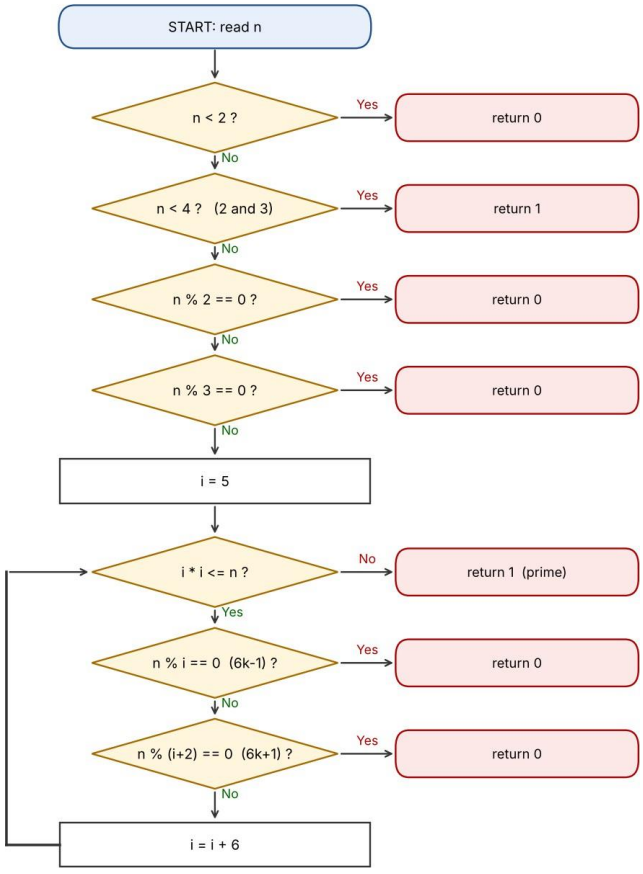


Figure 2: flowchart of the redesigned test

C.3 Extra memory and trade-offs

- **Extra memory:** none (two more comparisons, no arrays): O(1) space, same as the original.
- **Cost of the idea:** two divisions per pass share one loop test, so the test `i * i <= n` is made for i but the second candidate i+2 may lie just above $\sqrt{n}$. This is harmless (a divisor found there is still a genuine divisor of n, and i+2 = n is impossible for $n \geq 5$) but can cost one division beyond the original's bound.
- **Small inputs:** 2 and 3 are always tested, so for n = 5 and 7 the redesign uses 2 divisions while the original uses 1.
- **Complexity of the code:** 10 lines of pseudocode instead of 4; an off-by-two error (i += 6 versus i += 2) would silently break results, so validation against the original is part of the design.

- **Still O(√n):** the improvement is a constant factor (about 3), not a new complexity class; a sieve (Q5) changes the class when many tickets are checked, at the price of O(N) memory.

D. Prediction before implementing (1 mark)

Measure	Prediction (reasoned before running)	Basis
Divisions for n = 1000003	**334** (original 999; R1 predicted 500)	2 and 3, plus the numbers ≡ ±1 (mod 6) in [5, 1000]: 332 values (333 in [1,1000] minus the number 1). 332 + 2 = 334. R1: 1 + 499 odd values from 3 to 999 = 500.
Divisions for 7919	≈ 30	√7919 ≈ 88.99: 2, 3 + 28 values ±1 mod 6 in [5, 88]
Total, all tickets 2 … 1,000,003	**55 – 66% fewer**	primes (75% of the work) improve by about 2/3; composites that have factor 2 or 3 (about 72% of them) exit at the same step in both versions, so their share improves less
Running time	falls by about the same percentage as the divisions (≈ 60 – 65%)	the 64-bit remainder dominates the loop; the extra comparison is cheap
Correctness	0 mismatches	every prime factor candidate is retained

E. Implement and validate (2 marks)

Both versions (and R1) are in q7_redesign.c; they were run on the **same inputs** with the same division counter, compiled with gcc 13.3.0 `-Wall -Wextra -O2`.

E.1 Same inputs, division counts (original → redesigned)

n	Result	Original	R1 odd only	R2 6k±1	R2 / original
97	prime	8	5	4	0.50
100	composite	1	1	1	1.00
7919	prime	87	44	30	0.34
999983	prime	998	500	334	0.33
1000001	composite	100	51	35	0.35
1000003	prime	999	500	**334**	0.33
2, 3	prime	0	0	0	–
25, 35	composite	4	3	3	0.75
49	composite	6	4	4	0.67

E.2 Whole range, n = 2 … 1,000,003 (1,000,002 numbers, 78,499 primes)

Version	Total divisions	Primes only	Composites only	Worst single n	Time (best of 5)
Original	67,741,504	50,659,938	17,081,566	999	0.2442 s
R1 odd only	34,601,109 (−48.9%)	25,388,823	9,212,286	500	0.1268 s
R2 6k±1	**23,739,568 (−65.0%)**	17,030,426 (−66.4%)	6,709,142 (−60.7%)	**334**	**0.0852 s** (−65%)

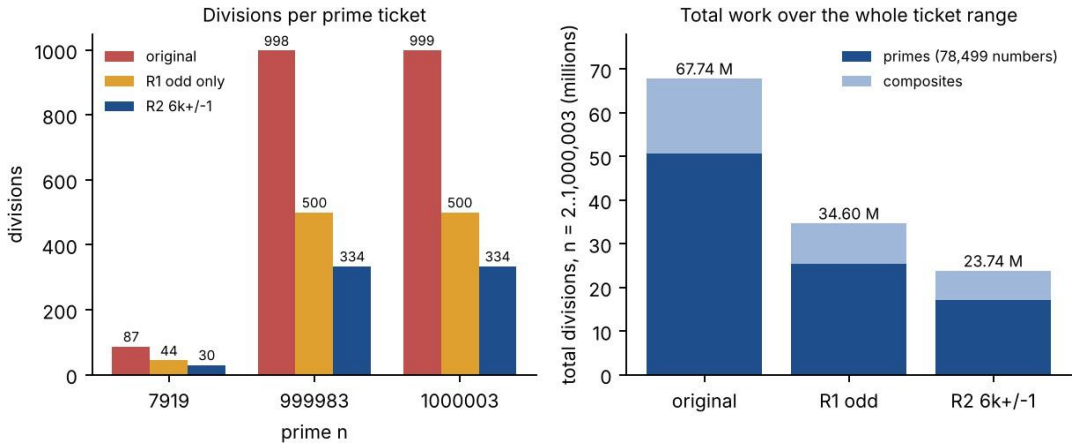


Figure 3: divisions per prime ticket and total work over the whole range (measured)

Validation. Results of the original, R1 and R2 were compared for every n from 2 to 1,000,003: **0 mismatches**, 78,499 primes in each case. Repeating the single check of 1000003 twenty thousand times took 0.0734 s (original), 0.0376 s (R1) and 0.0240 s (R2). A per-number comparison of the counters (q7_worse_check.c) gave:

```
R2 better on 333328 values, equal on 666672, worse on 2 (max extra divisions 1)
worse at n = 5 7
```

F. Defend the final design (2 marks)

Objective	Target	Measured	Met?
O1 divisions for 1000003	≤ 350	334 (999 → 334, −66.6%)	yes
O2 total work, 2 … 1,000,003	≥ 60% fewer	67,741,504 → 23,739,568 (−65.0%)	yes
O3 identical answers	0 mismatches	0 mismatches in 1,000,002 numbers	yes
O4 extra memory	none	none, O(1)	yes
O5 never noticeably worse	≤ original (+ small)	better on 333,328 numbers, equal on 666,672, worse on only 2 (n = 5 and 7) by 1 division	yes, with a stated exception

Prediction versus result. The division count for 1000003 (334) and for the per-ticket ratios matched the reasoned prediction exactly. The total reduction (65.0%) fell inside the predicted 55 – 66% band, near its top because primes dominate the work. The running time fell by 65%, in line with the prediction that the remainder instruction dominates.

Size of the improvement and new costs. A factor of about 2.9 fewer divisions overall and exactly 3.0 for large primes, at no memory cost. The costs are a longer routine, a second test per pass (which can examine one candidate slightly beyond $\sqrt{n}$), a fixed price of two divisions for the tiny primes 5 and 7, and a higher risk of off-by-two errors that must be caught by validation against the original.

Where the original may still be preferable. (1) When readability and verifiability matter more than a constant factor: the original is four lines and is easy to explain in a viva. (2) For a handful of tickets (a single check of 1000003 costs 3.7 µs in the original; the saving is not noticeable). (3) For tiny n (5 and 7) the original is one division cheaper. (4) When the workload is mostly even numbers, both stop at the first division and are identical (n = 100: 1 and 1).

Limits of the redesign and what would be next. The complexity class is still O($\sqrt{n}$); a wheel on 2, 3 and 5 would test 8 of every 30 numbers (theoretical 267 divisions for 1000003 – predicted only, not implemented), with diminishing returns for more code. When many tickets are checked, the Q5 sieve (O(N log log N) once, O(1) per query, O(N) memory) beats any trial-division variant: the measured break-even is about 31,300 random queries. The final recommendation is therefore: **use R2 (6k±1) for on-demand checks of individual tickets; build the sieve if the organisers verify the whole range or tens of thousands of tickets.**

Appendix A: source code and raw output (gcc 13.3.0)

q7_redesign.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

unsigned long checks;

/* ORIGINAL (Q5): trial division by every i with i <= n / i */
int prime_original(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* REDESIGN R1: test 2, then odd divisors only (the "improved" version of the Q5 screenshot) */
int prime_odd(unsigned long n){
    checks = 0; if (n < 2) return 0; if (n < 4) return 1;
    checks++; if (n % 2 == 0) return 0;
    for (unsigned long i = 3; i <= n / i; i += 2) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* REDESIGN R2: test 2 and 3, then only i = 6k-1 and 6k+1 */
int prime_6k(unsigned long n){
    checks = 0; if (n < 2) return 0; if (n < 4) return 1;
    checks++; if (n % 2 == 0) return 0;
    checks++; if (n % 3 == 0) return 0;
    for (unsigned long i = 5; i <= n / i; i += 6) {
        checks++; if (n % i == 0)      return 0;   /* 6k - 1 */
        checks++; if (n % (i + 2) == 0) return 0;    /* 6k + 1 */
    }
    return 1;
}
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec + t.tv_nsec*1e-9;
     }

int main(void){
    const unsigned long N = 1000003;
    unsigned long t[] = {97, 100, 7919, 999983, 1000001, 1000003, 2, 3, 25, 49, 35};
    int nt = sizeof t / sizeof t[0];
    printf("== Same inputs, divisions performed ==\n");
    printf("%-9s %-10s %9s %9s %9s %9s\n","n","result","original","R1 odd","R2 6k+-1","R2/orig");
    for (int k = 0; k < nt; k++){
        unsigned long n = t[k];
        int a = prime_original(n); unsigned long ca = checks;
        int b = prime_odd(n);      unsigned long cb = checks;
        int c = prime_6k(n);       unsigned long cc = checks;
        printf("%-9lu %-10s %9lu %9lu %9lu %8.2f %s\n", n, a?"prime":"composite", ca, cb, cc,
               ca ? (double)cc/ca : 0.0, (a==b && b==c) ? "" : "MISMATCH");
    }

    /* full validation + totals, split by prime / composite */
    unsigned long tot[3] = {0,0,0}, totP[3] = {0,0,0}, totC[3] = {0,0,0}, bad = 0, np = 0;
    unsigned long worst[3] = {0,0,0};
    for (unsigned long n = 2; n <= N; n++){
        int r0 = prime_original(n); unsigned long c0 = checks;
        int r1 = prime_odd(n);      unsigned long c1 = checks;
        int r2 = prime_6k(n);       unsigned long c2 = checks;
        if (r0 != r1 || r0 != r2) bad++;
        tot[0]+=c0; tot[1]+=c1; tot[2]+=c2;
        if (r0){ np++; totP[0]+=c0; totP[1]+=c1; totP[2]+=c2; } else { totC[0]+=c0; totC[1]+=c1; totC[2]+=c2; }
        if (c0>worst[0]) worst[0]=c0;
        if (c1>worst[1]) worst[1]=c1;
        if (c2>worst[2]) worst[2]=c2;
    }
    printf("\n== Every n = 2..%lu (mismatches between versions: %lu; primes found: %lu) ==\n", N, bad, np);
    const char *nm[3] = {"original","R1 odd","R2 6k+-1"};
    printf("%-10s %14s %14s %14s %10s\n","version","all n","primes only","composites","worst case");
    for (int v = 0; v < 3; v++) printf("%-10s %14lu %14lu %14lu %10lu\n", nm[v], tot[v], totP[v], totC[v],
        worst[v]);
    for (int v = 1; v < 3; v++)
        printf("%s vs original: all n = %.1f%% fewer, primes only = %.1f%% fewer, composites = %.1f%% fewer\n",
            nm[v],
               100.0*(1.0-(double)tot[v]/tot[0]), 100.0*(1.0-(double)totP[v]/totP[0]),
                   100.0*(1.0-(double)totC[v]/totC[0]));

    /* timing: all n up to N, best of 5 */
    printf("\n== Timing for n = 2..%lu (best of 5, -O2) ==\n", N);
    for (int v = 0; v < 3; v++){
        double best = 1e9; unsigned long sink = 0;
        for (int rep = 0; rep < 5; rep++){
            double t0 = now(); unsigned long cnt = 0;
            for (unsigned long n = 2; n <= N; n++)
```

```
                cnt += (v==0) ? prime_original(n) : (v==1) ? prime_odd(n) : prime_6k(n);
            double dt = now() - t0; if (dt < best) best = dt; sink = cnt;
        }
        printf("%-10s %.4f s (primes counted = %lu)\n", nm[v], best, sink);
    }
    /* timing: large primes only (worst case) */
    printf("\n== Timing: 20000 repeated checks of the prime 1000003 ==\n");
    for (int v = 0; v < 3; v++){
        double t0 = now(); unsigned long cnt = 0;
        for (int rep = 0; rep < 20000; rep++)
            cnt += (v==0) ? prime_original(1000003) : (v==1) ? prime_odd(1000003) : prime_6k(1000003);
        printf("%-10s %.4f s (sink %lu)\n", nm[v], now()-t0, cnt);
    }
    /* repeated work: how often small divisors are retried */
    unsigned long by2=0, by3=0, byAny=0;
    /* in original: number of divisions with i even, or i multiple of 3, across all primes */
    for (unsigned long n = 2; n <= N; n++){
        int p = 1; for (unsigned long i = 2; i <= n / i; i++){ if (n % i == 0){ p = 0; break; }
            if (i % 2 == 0) by2++; else if (i % 3 == 0) by3++; byAny++; }
        (void)p;
    }
    printf("\nOriginal, divisions that did NOT end the loop: total=%lu, with i even=%lu (%.1f%%), with i odd
        multiple of 3=%lu (%.1f%%)\n",
           byAny, by2, 100.0*by2/byAny, by3, 100.0*by3/byAny);
    return 0;
}
```

Raw output of q7_redesign

```
== Same inputs, divisions performed ==
n          result       original     R1 odd   R2 6k+-1   R2/orig
97         prime               8          5          4      0.50
100        composite           1          1          1      1.00
7919       prime              87         44         30      0.34
999983     prime             998        500        334      0.33
1000001    composite         100         51         35      0.35
1000003    prime             999        500        334      0.33
2          prime               0          0          0      0.00
3          prime               0          0          0      0.00
25         composite           4          3          3      0.75
49         composite           6          4          4      0.67
35         composite           4          3          3      0.75

== Every n = 2..1000003 (mismatches between versions: 0; primes found: 78499) ==
version             all n     primes only     composites worst case
original          67741504        50659938        17081566         999
R1 odd            34601109        25388823         9212286         500
R2 6k+-1          23739568        17030426         6709142         334
R1 odd vs original: all n = 48.9% fewer, primes only = 49.9% fewer, composites = 46.1% fewer
R2 6k+-1 vs original: all n = 65.0% fewer, primes only = 66.4% fewer, composites = 60.7% fewer

== Timing for n = 2..1000003 (best of 5, -O2) ==
original   0.2442 s (primes counted = 78499)
R1 odd     0.1268 s (primes counted = 78499)
R2 6k+-1   0.0852 s (primes counted = 78499)

== Timing: 20000 repeated checks of the prime 1000003 ==
original   0.0734 s (sink 20000)
R1 odd     0.0376 s (sink 20000)
R2 6k+-1   0.0240 s (sink 20000)

Original, divisions that did NOT end the loop: total=66820001, with i even=33640395 (50.3%), with i odd multiple
     of 3=11220986 (16.8%)
```

q7_worse_check.c

```
#include <stdio.h>
unsigned long checks;
int po(unsigned long n){checks=0; if(n<2)return 0; for(unsigned long i=2;i<=n/i;i++){checks++; if(n%i==0)return
    0;} return 1;}
int p6(unsigned long n){checks=0; if(n<2)return 0; if(n<4)return 1; checks++; if(n%2==0)return 0; checks++;
    if(n%3==0)return 0;
  for(unsigned long i=5;i<=n/i;i+=6){checks++; if(n%i==0)return 0; checks++; if(n%(i+2)==0)return 0;} return 1;}
int main(void){ unsigned long N=1000003, worse=0, same=0, better=0; long maxextra=0; unsigned long first[20];
    int nf=0;
 for(unsigned long n=2;n<=N;n++){ po(n); unsigned long a=checks; p6(n); unsigned long b=checks;
  if(b>a){worse++; if(nf<20) first[nf++]=n; if((long)(b-a)>maxextra) maxextra=b-a;} else if(b==a) same++; else
      better++; }
 printf("R2 better on %lu values, equal on %lu, worse on %lu (max extra divisions %ld)\nworse at n =", better,
     same, worse, maxextra);
 for(int k=0;k<nf;k++) printf(" %lu", first[k]); printf("\n"); return 0; }
```